

Theory of Computation + NP-Completeness - Placement Interview Guide

Two clusters in one guide:

- **Part A** - Classic NP problems (SAT ... Graph Coloring) with the reduction web that ties them together.
- **Part B** - The most-asked conceptual TOC questions (DFA vs NFA ... why SAT matters).

Read Part A's shared framework first; every problem after it reuses those exact ideas.

PART A - THE NP PROBLEMS

A0. Shared Framework (read this once, reuse everywhere)

Before the individual problems, lock these definitions. Every problem below is an instance of this framework, so you only learn it once.

Decision vs Optimization

- **Decision problem:** answer is **YES / NO**. ("Is there a clique of size $\geq k$?")
- **Optimization problem:** answer is a **number or structure**. ("What is the largest clique?")
- NP-completeness is defined only for **decision problems**. We convert optimization $\rightarrow$ decision by adding a threshold k .

The complexity classes

Class	Plain meaning
P	Solvable in polynomial time (n^c). "Easy."
NP	A given YES-answer can be verified in polynomial time using a <i>certificate</i> (a proposed solution).
NP-hard	At least as hard as every problem in NP. Everything in NP reduces to it. May not be in NP.
NP-complete	In NP and NP-hard. The hardest problems that are still verifiable in poly time.

Verifier intuition (the heart of NP)

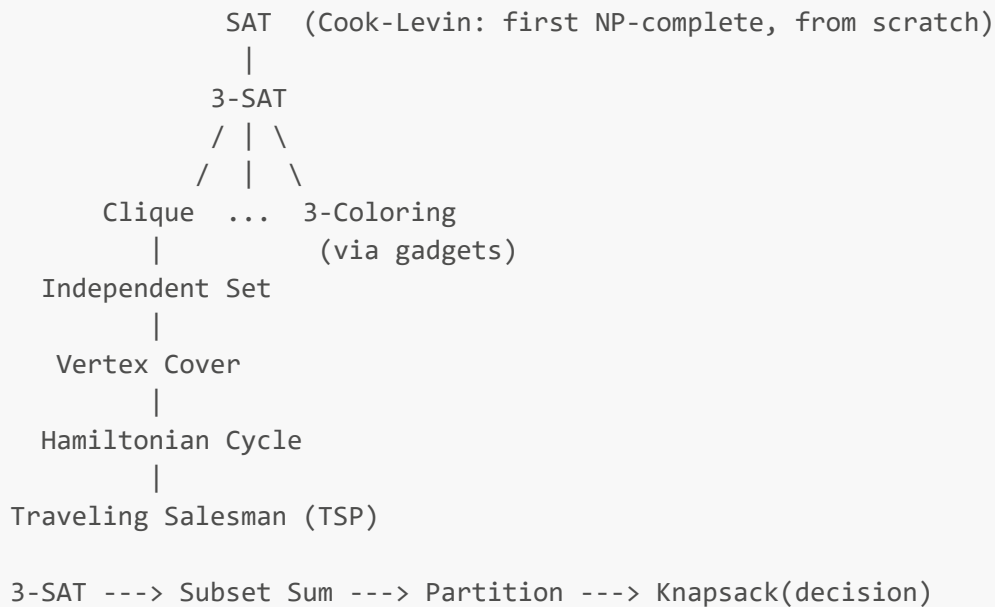
NP = "hard to *find* an answer, easy to *check* one." Give me a proposed clique of size $k \rightarrow$ I check all pairs of its k vertices in $O(k^2) \rightarrow$ YES/NO fast. Finding it is the hard part.

Polynomial reduction ($A \leq_p B$)

"Transform any instance of A into an instance of B in poly time, so that A is YES iff B is YES."

- If $A \leq_p B$ and B is easy, then A is easy.
- If $A \leq_p B$ and A is hard, then B is hard.
- To prove **B is NP-complete**: (1) show B in NP, (2) reduce a known NP-complete problem A to B.

The reduction web (memorize this picture)



Arrows mean "reduces to" (source is used to prove target is NP-complete).

Quick map of the 11 problems

Problem	One-line	NP-complete?
SAT	Is a boolean formula satisfiable?	Yes (the original)
3-SAT	SAT with exactly 3 literals per clause	Yes
Clique	k mutually-connected vertices?	Yes
Independent Set	k mutually- non -connected vertices?	Yes
Vertex Cover	k vertices touching every edge?	Yes
Hamiltonian Cycle	Cycle visiting every vertex once?	Yes
TSP (decision)	Tour of length $\leq k$ visiting all cities?	Yes
Subset Sum	Subset summing to exactly T?	Yes
Partition	Split into two equal-sum halves?	Yes
Knapsack (decision)	Value $\geq V$ within weight W?	Yes (weakly NP-complete)
Graph Coloring	Proper coloring with k colors?	Yes ($k \geq 3$)

"Weakly NP-complete" = has a pseudo-polynomial DP (poly in the numeric values, not their bit-length).
 Subset Sum, Partition, Knapsack are all in this family.

A1. SAT (Boolean Satisfiability)

1. Overview

- **Definition:** Given a boolean formula over variables $x_1 \dots x_n$ (using AND, OR, NOT), is there an assignment of TRUE/FALSE that makes the whole formula TRUE?
- **Why it matters:** It is the *first* problem ever proved NP-complete (Cook-Levin theorem, 1971). It is the "root" every other NP-complete proof hangs from.
- **Where used in real systems:** SAT solvers power hardware verification (chip checking), software model checking, AI planning, package dependency resolvers (**apt**, **npm**), and cryptanalysis.
- **Why interviewers ask:** It tests whether you understand *why* a whole class of problems is hard, plus the verifier idea.

2. Core Idea

- **Intuition:** n variables $\rightarrow 2^n$ possible assignments. Brute force is exponential. But if someone *hands* you an assignment, you plug it in and evaluate in linear time - that is the NP verifier.
- **Real-world analogy:** A lock with n switches. Trying every combination is slow; checking whether a given combination opens it is instant.
- **Small example:** $(x_1 \text{ OR } x_2) \text{ AND } (\text{NOT } x_1 \text{ OR } x_3)$. Assignment $x_1=F, x_2=T, x_3=\text{anything}$ $\rightarrow$ first clause T, second clause T $\rightarrow$ **satisfiable**.
- **Step-by-step verify:** substitute values $\rightarrow$ evaluate each clause $\rightarrow$ AND them $\rightarrow$ TRUE means the certificate is valid.

3. Important Subtopics

- **CNF (Conjunctive Normal Form):** AND of clauses, each clause an OR of literals. Standard input form. *Angle:* any formula converts to CNF; 3-SAT restricts each clause to 3 literals.
- **Certificate / witness:** the satisfying assignment itself. *Why:* it is the proof that SAT is in NP.
- **Cook-Levin theorem:** any NP problem's Turing-machine run is *encoded* as a SAT formula. *Angle:* "How was the first NP-complete problem proven with nothing to reduce from?" $\rightarrow$ by encoding the generic NP verifier directly.
- **DPLL / CDCL:** real solver algorithms (backtracking + clause learning). *Angle:* "SAT is NP-complete, why do solvers work?" $\rightarrow$ worst case is exponential, but real instances have structure.

4. Real-World Example Package managers: "Can I install these packages so all version constraints hold?" is literally a SAT instance. **apt/dnf** and many build systems embed a SAT/SMT solver.

5. Mental Model

```
Formula ----(hard: find)----> satisfying assignment?
Formula <---(easy: check)----- given assignment
NP = the gap between these two arrows.
```

6. Common Interview Questions

1. *What is SAT?* -> Deciding if a boolean formula has a satisfying assignment. Key: it is a **decision** problem. Mistake: describing it as "solve equations."
2. *Why is SAT in NP?* -> A satisfying assignment is a poly-checkable certificate. Mistake: confusing "solve fast" with "verify fast."
3. *What is Cook-Levin?* -> SAT is NP-complete; every NP problem reduces to it. Mistake: forgetting it was proven from the definition of NP, not another NPC problem.
4. *Is SAT in P?* -> Unknown; equivalent to P vs NP. Mistake: saying "no" as if proven.
5. *What is CNF?* -> AND of OR-clauses. Mistake: mixing CNF (AND of ORs) with DNF (OR of ANDs).
6. *Is 2-SAT NP-complete?* -> No! 2-SAT is in **P** (implication graph / SCCs). Big trap.
7. *Why do solvers work if SAT is NP-complete?* -> NP-completeness is worst-case; real formulas are structured, CDCL prunes hard.
8. *SAT vs 3-SAT?* -> 3-SAT restricts clauses to exactly 3 literals; still NP-complete, easier to reduce from.
9. *Give a SAT and an UNSAT formula.* -> SAT: x . UNSAT: $x \text{ AND NOT } x$.
10. *How does SAT prove other problems hard?* -> Reduce 3-SAT to them via gadgets.

7. Deep-Dive Questions

1. *Why is 2-SAT in P but 3-SAT NP-complete?* -> 2-SAT clauses become implications ($a \text{ OR } b = !a \rightarrow b$); satisfiable iff no variable and its negation share an SCC. 3 literals can't be captured by simple implications.
2. *What does the Cook-Levin reduction encode?* -> The tableau of a nondeterministic TM run: variables for tape/head/state per step, clauses enforcing valid transitions and an accepting run.
3. *Is Horn-SAT easy?* -> Yes, linear time via unit propagation.
4. *SAT vs SMT?* -> SMT adds theories (arithmetic, arrays); strictly more expressive, used in program verification.
5. *What is #SAT?* -> Counting satisfying assignments; #P-complete, believed harder than SAT.

8. Comparison Table

	SAT	2-SAT	3-SAT	Horn-SAT
Literals/clause	any	2	3	any, ≤ 1 positive
Complexity	NP-complete	P	NP-complete	P (linear)
Method	DPLL/CDCL	SCC on implication graph	DPLL/CDCL	unit propagation

9. Common Mistakes

- Thinking 2-SAT is NP-complete (it is in P).
- Saying SAT is "unsolvable" - it is decidable, just not known to be fast.
- Confusing CNF and DNF (DNF *satisfiability* is easy; DNF *validity* is hard).

10. Edge Cases

- Empty formula (no clauses) = trivially TRUE.
- A single empty clause = FALSE (unsatisfiable).
- DNF-SAT is in P; hardness is specific to CNF.

11. How to Explain in Interview "SAT asks whether a boolean formula can be made true. It is the first NP-complete problem - Cook-Levin showed every NP problem reduces to it. Canonical 'hard to solve, easy to

check': given an assignment you verify in linear time, but finding one may need exponential search. Everything else NP-complete is proven hard by reducing SAT (usually 3-SAT) to it."

12. Quick Revision

- SAT = satisfiability of a boolean CNF formula. Decision problem.
- First NP-complete (Cook-Levin, 1971). In NP: assignment is the certificate.
- 2-SAT and Horn-SAT are in P; 3-SAT is NP-complete.
- Trap: "verify fast" != "solve fast."

13. Practice Tasks

- By hand, decide satisfiability of $(x \text{ OR } y) \text{ AND } (\text{NOT } x \text{ OR NOT } y) \text{ AND } (x \text{ OR NOT } y)$.
- Write a brute-force SAT checker in Python (try all 2^n assignments).
- Convert $(a \rightarrow b)$ and $(a \text{ XOR } b)$ into CNF.

14. Cheat Sheet

- **Def:** Is a boolean formula satisfiable?
- **Why:** First NP-complete; root of all reductions.
- **Most asked:** Cook-Levin, NP verifier, 2-SAT is P.
- **Compare:** SAT(NPC) vs 2-SAT(P) vs DNF-SAT(P).
- **One-liner:** "The original NP-complete problem: hard to satisfy, trivial to verify."

A2. 3-SAT

1. Overview

- **Definition:** SAT in CNF where **every clause has exactly 3 literals**. Is it satisfiable?
- **Why it matters:** Standard *starting point* for reduction proofs - its rigid 3-literal shape makes building "gadgets" clean.
- **Where used:** Same solver ecosystem as SAT; primarily a theoretical workhorse for proving NP-completeness of graph/number problems.
- **Why interviewers ask:** Explaining a 3-SAT $\rightarrow$ X reduction proves you understand NP-completeness, not just definitions.

2. Core Idea

- **Intuition:** Restricting to 3 literals loses no power (still NP-complete) but gains uniform structure to build gadgets from.
- **Analogy:** Standardizing every Lego brick to the same size so you can build predictable machines from them.
- **Example:** $(x_1 \text{ OR } x_2 \text{ OR NOT } x_3) \text{ AND } (\text{NOT } x_1 \text{ OR } x_2 \text{ OR } x_3)$. Satisfiable by $x_2 = \text{True}$.
- **Step-by-step:** any clause with $k > 3$ literals is split using new variables; $k < 3$ is padded by repeating a literal. Result is 3-SAT-equivalent.

3. Important Subtopics

- **Clause-to-3 conversion:** $(a \text{ OR } b \text{ OR } c \text{ OR } d) \rightarrow (a \text{ OR } b \text{ OR } z) \text{ AND } (\text{NOT } z \text{ OR } c \text{ OR } d)$. *Angle:* "Turn general SAT into 3-SAT."

- **Gadget-based reductions:** variable gadgets + clause gadgets wired together. *Angle:* draw the Clique / Independent-Set gadget.
- **2-SAT boundary:** 3 is the exact threshold where it flips from P to NP-complete. *Angle:* "Why 3 and not 2?"

4. Real-World Example Automated planning/AI: "Is there a valid plan of length L?" is encoded as a 3-SAT (or general SAT) instance and handed to a solver.

5. Mental Model

```
General SAT --(clause splitting)--> 3-SAT (same difficulty)
3-SAT --(gadgets)--> Clique / IndSet / VertexCover / 3-Coloring / SubsetSum
```

6. Common Interview Questions

1. *What is 3-SAT?* -> CNF SAT, exactly 3 literals/clause. Mistake: "at most 3" - it is exactly 3 (padding fixes shorter ones).
2. *Is 3-SAT NP-complete?* -> Yes. Mistake: assuming the restriction makes it easier.
3. *Why start reductions from 3-SAT?* -> Uniform clause size = clean gadgets.
4. *Convert a 4-literal clause to 3-SAT.* -> Show the splitting trick with a fresh variable.
5. *Why 2-SAT in P but 3-SAT NP-complete?* -> 2 literals = implications (SCC solvable); 3 breaks that.
6. *Sketch 3-SAT -> Clique.* -> One vertex per literal-occurrence, group per clause; edge between literals in different clauses that don't conflict; clique of size (#clauses) = satisfying assignment.
7. *Does 3-SAT lose power vs SAT?* -> No, poly-equivalent.
8. *What is a gadget?* -> A small fixed structure encoding a variable or clause inside the target problem.
9. *Is 1-SAT hard?* -> No, forced assignments solve it.
10. *Give an UNSAT 3-SAT.* -> All 8 clauses over 3 variables (every combination forbidden).

7. Deep-Dive Questions

1. *Why exactly 3?* -> 2-SAT is implication-closed (P); at 3 you can encode arbitrary constraints, enough for hardness.
2. *MAX-3-SAT approximability?* -> Random assignment satisfies 7/8 of clauses in expectation, and 7/8 is optimal unless P=NP (Hastad).
3. *Detail 3-SAT -> Independent Set.* -> Triangle per clause (3 literal-vertices) + edges between contradictory literals; independent set of size m (m=#clauses) iff satisfiable.
4. *Is planar 3-SAT NP-complete?* -> Yes; used for geometric hardness proofs.
5. *Why does clause padding preserve satisfiability?* -> Repeating a literal doesn't change the clause's truth value.

8. Comparison Table

	SAT	3-SAT	2-SAT
Clause width	any	exactly 3	exactly 2
Complexity	NP-complete	NP-complete	P
Role	first NPC	reduction source	tractable boundary

9. Common Mistakes

- "3-SAT is easier than SAT" - same class.
- Reading "3" as "at most 3" instead of the canonical "exactly 3."
- Thinking conversion to 3-SAT blows up exponentially - it is linear.

10. Edge Cases

- Clauses with a repeated literal ($x \text{ OR } x \text{ OR } y$) are legal padding.
- Random 3-SAT has a sharp satisfiability threshold near clause/variable ratio ~ 4.27 .

11. How to Explain in Interview "3-SAT is SAT restricted to exactly three literals per clause. It stays NP-complete, and its uniform shape makes it the go-to source for reduction proofs. Two literals per clause would be polynomial - 3 is the exact tipping point into NP-completeness."

12. Quick Revision

- 3 literals/clause, still NP-complete. Standard reduction source.
- 2-SAT is P; 3 is the boundary.
- MAX-3-SAT: 7/8 approximation is optimal unless $P=NP$.

13. Practice Tasks

- Convert ($a \text{ OR } b \text{ OR } c \text{ OR } d \text{ OR } e$) into 3-SAT clauses.
- Draw the 3-SAT $\rightarrow$ Independent Set reduction for a 2-clause formula.
- Implement a 3-SAT brute forcer; test the 4.27 threshold empirically.

14. Cheat Sheet

- **Def:** Satisfiability with exactly 3 literals per clause.
- **Why:** Cleanest reduction source for NP-completeness proofs.
- **Most asked:** why 3 not 2; a 3-SAT $\rightarrow$ graph reduction.
- **Compare:** 2-SAT(P) vs 3-SAT(NP).
- **One-liner:** "SAT in standard 3-literal form - the launchpad for every hardness proof."

A3. Clique

1. Overview

- **Definition:** In an undirected graph G , a **clique** is a set of vertices that are **all pairwise connected**.
Decision: does G contain a clique of size $\geq k$?
- **Why it matters:** Classic NP-complete graph problem; models "everyone knows everyone" groups.
- **Where used:** Social network community detection, bioinformatics (protein interaction clusters), finding fully-connected sub-modules.
- **Why interviewers ask:** Tests graph reasoning plus the Clique / Independent-Set / Vertex-Cover triangle of equivalences.

2. Core Idea

- **Intuition:** A clique is a "perfect friend group" - every pair is directly linked. Checking a proposed group is easy (verify all pairs); finding the biggest is hard.

- **Analogy:** A group of people where every single pair are direct friends (no friend-of-a-friend gaps).
- **Example:** Vertices {A,B,C} with edges AB, BC, CA form a 3-clique (triangle). Add D linked to all three -> 4-clique.
- **Step-by-step verify:** given a candidate set S of size k, check every one of the $C(k,2)$ pairs has an edge -> YES/NO in poly time.

3. Important Subtopics

- **Clique vs Independent Set:** a clique in G = an independent set in the **complement** graph G'. *Angle:* "Relate Clique and Independent Set."
- **Max-Clique (optimization):** largest clique; NP-hard, also hard to approximate.
- **3-SAT -> Clique reduction:** literal-vertices grouped per clause, edges between compatible literals of different clauses; k = #clauses. *Angle:* "Prove Clique NP-complete."

4. Real-World Example Social graphs: finding the largest set of users who all mutually follow each other (a tight community) is a max-clique query.

5. Mental Model

```

Clique in G  <==complement==>  Independent Set in G'
    |                               |
    +----- Vertex Cover = V minus Independent Set

```

6. Common Interview Questions

1. *What is a clique?* -> Fully-connected vertex subset. Mistake: confusing with connected component.
2. *Is Clique NP-complete?* -> Yes. In NP (verify pairs), NP-hard (from 3-SAT).
3. *Relate Clique and Independent Set.* -> Clique in G = IndSet in complement G'.
4. *How to verify a k-clique?* -> Check all $C(k,2)$ pairs are edges.
5. *Reduce 3-SAT to Clique.* -> Vertex per literal-occurrence, edges between non-conflicting literals of different clauses, want clique of size #clauses.
6. *Is finding a triangle (3-clique) hard?* -> No - fixed k makes it polynomial ($O(n^3)$ or better). Hardness needs k as input.
7. *Max-Clique vs Clique decision?* -> Optimization vs YES/NO threshold form.
8. *Complement graph meaning?* -> Same vertices, edges flipped.
9. *Can Clique be approximated well?* -> No; hard to approximate within $n^{(1-\epsilon)}$.
10. *Clique in a complete graph?* -> The whole vertex set is one big clique.

7. Deep-Dive Questions

1. *Why is fixed-k Clique in P but general Clique NP-complete?* -> Fixed k -> $O(n^k)$ is polynomial; when k is part of input, k can be $\sim n/2$, giving exponential search.
2. *Detail the 3-SAT -> Clique reduction correctness.* -> A size-m clique picks one true literal per clause with no contradictions -> consistent satisfying assignment, and vice versa.
3. *What is the clique cover problem?* -> Partition vertices into fewest cliques; also NP-hard.
4. *Is Clique fixed-parameter tractable in k?* -> No (W[1]-complete), unlike Vertex Cover which is FPT.
5. *Hardness of approximation?* -> Cannot approximate max-clique within $n^{(1-\epsilon)}$ unless P=NP.

8. Comparison Table

Concept	Every pair...	Lives in
Clique	connected	G
Independent Set	not connected	G (= clique in G')
Vertex Cover	complement of IndSet	V minus IndSet

9. Common Mistakes

- Confusing clique (all pairs edges) with connected component (path exists).
- Thinking triangle-finding proves Clique is hard (fixed k is easy).
- Forgetting the complement relationship to Independent Set.

10. Edge Cases

- $k=1$: any single vertex is a clique.
- $k=2$: any edge is a clique.
- Empty graph: max clique = 1.

11. How to Explain in Interview "A clique is a subset of vertices that are all mutually adjacent. Deciding if a k -clique exists is NP-complete - easy to verify, hard to find. It is equivalent to Independent Set on the complement graph, which ties it to Vertex Cover too."

12. Quick Revision

- Clique = fully-connected subset. Decision: k -clique exists?
- NP-complete (3-SAT reduction). Fixed k is polynomial.
- Clique in G = IndSet in complement.
- Hard to approximate.

13. Practice Tasks

- Find the max clique in a 6-vertex graph by hand.
- Write brute-force max-clique (check all subsets) in Python.
- Given a graph, build its complement and find an independent set = clique.

14. Cheat Sheet

- **Def:** Subset where all pairs are adjacent; k -clique exists?
- **Why:** Core NP-complete graph problem; community detection.
- **Most asked:** complement link to IndSet; fixed- k is easy.
- **Compare:** Clique vs IndSet vs VertexCover triangle.
- **One-liner:** "Everyone-knows-everyone subset - equivalent to an independent set in the complement graph."

A4. Independent Set

1. Overview

- **Definition:** A set of vertices with **no edges between any two of them**. Decision: does G have an independent set of size $\geq k$?
- **Why it matters:** Dual of Clique; models "conflict-free selection."
- **Where used:** Scheduling non-conflicting tasks, register allocation, wireless channel assignment, selecting mutually compatible items.
- **Why interviewers ask:** The Clique/IndSet/VertexCover equivalence is a favorite "connect the dots" question.

2. Core Idea

- **Intuition:** Pick as many vertices as possible such that none of them clash (no shared edge). It is the exact opposite of a clique.
- **Analogy:** Inviting guests where no two invitees dislike each other (edges = dislikes).
- **Example:** Path A-B-C-D. $\{A, C\}$ is independent (no edge). $\{A, C\}$ or $\{B, D\}$ are max independent sets of size 2.
- **Step-by-step verify:** given candidate S , confirm no pair in S has an edge.

3. Important Subtopics

- **Complement of Vertex Cover:** S is an independent set **iff** V minus S is a vertex cover. *Angle:* "Relate IndSet and Vertex Cover" - they are exact complements.
- **IndSet = Clique in complement.** *Angle:* chain all three problems.
- **Max Independent Set:** NP-hard optimization; hard to approximate.

4. Real-World Example Register allocation in compilers: variables are vertices, edges mean "live at the same time." A large independent set = variables that can share resources without conflict (though allocation uses the coloring view).

5. Mental Model

Independent Set $S \iff$ Vertex Cover $(V \setminus S)$
 $\quad \quad \quad \downarrow$
 $\quad \quad \quad = \text{Clique in complement graph}$
 $\text{Max IndSet} + \text{Min VertexCover} = |V|$

6. Common Interview Questions

1. What is an independent set? -> Vertices with no edges among them. Mistake: allowing one edge.
2. Is IndSet NP-complete? -> Yes (equivalent to Clique/VertexCover).
3. Relate IndSet and Vertex Cover. -> Complements: S independent iff $V \setminus S$ is a cover.
4. Relate IndSet and Clique. -> IndSet in $G =$ Clique in complement G' .
5. Verify an independent set? -> Check no pair has an edge.
6. $\text{Max IndSet} + \text{Min VC} = ?$ -> $|V|$. Key identity.
7. Is IndSet on trees hard? -> No - polynomial via DP on trees (special case).
8. Independent set in a complete graph? -> Size 1 only.
9. Why is it NP-complete if Vertex Cover is FPT? -> Complement relation preserves the decision but not parameter tractability in k .
10. Approximable? -> Hard, like Clique.

7. Deep-Dive Questions

1. *Prove $\text{Max IndSet} + \text{Min VC} = |V|$.* -> The complement of any maximal independent set is a minimal vertex cover, and vice versa.
2. *Why is IndSet easy on trees/bipartite-ish structures?* -> Tree DP (include/exclude each node) runs in linear time; bipartite uses Konig's theorem ($\text{VC} = \text{max matching}$).
3. *Is IndSet FPT in k ?* -> No ($W[1]$ -hard), same as Clique.
4. *Weighted Independent Set?* -> Maximize total weight; still NP-hard on general graphs, poly on trees.
5. *Connection to graph coloring?* -> Each color class is an independent set; coloring partitions V into independent sets.

8. Comparison Table

	Independent Set	Clique	Vertex Cover
Definition	no internal edges	all internal edges	touches every edge
Relation	base	IndSet in complement	V minus IndSet
Goal	maximize	maximize	minimize
On trees	P	P	P

9. Common Mistakes

- Thinking a maximal (can't extend) set equals a maximum (largest) set - not the same.
- Forgetting the exact complement identity with Vertex Cover.
- Assuming it is hard on all graphs (trees/bipartite are easy).

10. Edge Cases

- Empty graph: all vertices form one independent set.
- Complete graph: max independent set = 1.
- Isolated vertices always join an independent set.

11. How to Explain in Interview "An independent set is a group of mutually non-adjacent vertices. Its complement is exactly a vertex cover, and it equals a clique in the complement graph - so these three problems are all NP-complete together. $\text{Max-IndSet} + \text{Min-Vertex-Cover}$ always equals the number of vertices."

12. Quick Revision

- IndSet = no edges inside the set. NP-complete.
- Complement of Vertex Cover; clique in complement graph.
- $\text{Max IndSet} + \text{Min VC} = |V|$.
- Poly on trees/bipartite.

13. Practice Tasks

- Compute max independent set of a 5-node cycle by hand (answer: 2).
- Write tree-DP for max weighted independent set.
- Verify the complement identity on a small graph.

14. Cheat Sheet

- **Def:** Largest set of pairwise non-adjacent vertices.
 - **Why:** Conflict-free selection; dual of Clique and VC.
 - **Most asked:** complement of Vertex Cover; = Clique in complement.
 - **Compare:** IndSet vs Clique vs VC.
 - **One-liner:** "Pick vertices that never clash - the exact complement of a vertex cover."
-

A5. Vertex Cover

1. Overview

- **Definition:** A set of vertices such that **every edge has at least one endpoint in the set**. Decision: is there a vertex cover of size $\leq k$?
- **Why it matters:** One of Karp's original 21 NP-complete problems; also the poster child for approximation and FPT algorithms.
- **Where used:** Network monitoring (place monitors to watch every link), conflict resolution, bioinformatics.
- **Why interviewers ask:** Rich problem - NP-complete yet has a 2-approximation and is fixed-parameter tractable, a great discussion of "hard but tractable in practice."

2. Core Idea

- **Intuition:** Put guards on vertices so every edge (corridor) is watched by at least one guard. Minimize guards.
- **Analogy:** Cameras at street intersections so every street is covered by at least one camera.
- **Example:** Star graph (center C connected to leaves) - {C} alone covers all edges $\rightarrow$ vertex cover size 1.
- **Step-by-step verify:** given set S, check every edge has an endpoint in S.

3. Important Subtopics

- **Complement of Independent Set:** min vertex cover = V minus max independent set. *Angle:* the identity again.
- **2-approximation:** repeatedly pick any uncovered edge, add **both** endpoints. Guarantees $\leq 2 \times$ optimal. *Angle:* "Give an approximation algorithm."
- **FPT algorithm:** $O(2^k * n)$ - branch on each edge (one of two endpoints must be in the cover). *Angle:* "Vertex Cover with small k is tractable."
- **Konig's theorem:** in bipartite graphs, min vertex cover = max matching (polynomial). *Angle:* special tractable case.

4. Real-World Example Network security: place intrusion monitors on the fewest routers so that every network link passes through a monitored router = minimum vertex cover.

5. Mental Model

Every edge must have ≥ 1 endpoint chosen.
 Min Vertex Cover = $|V|$ - Max Independent Set
 2-approx: grab both ends of any uncovered edge, repeat.

6. Common Interview Questions

1. *What is a vertex cover?* -> Vertex set touching every edge. Mistake: "covers every vertex" (it is edges).
2. *Is VC NP-complete?* -> Yes (from 3-SAT / Independent Set).
3. *Relate VC and Independent Set.* -> Exact complements; $\text{MinVC} + \text{MaxIS} = |V|$.
4. *Give a 2-approximation.* -> Pick both endpoints of any uncovered edge, repeat; $\leq 2 \times$ optimal.
5. *Why does that give 2x?* -> Each chosen edge needs ≥ 1 endpoint in OPT; we take 2, so at most double.
6. *Is VC fixed-parameter tractable?* -> Yes, $O(2^k n)$ - great for small k .
7. *VC on bipartite graphs?* -> Polynomial via Konig (= max matching).
8. *VC vs Dominating Set?* -> VC covers edges; dominating set covers vertices (every vertex in or adjacent to the set).
9. *Verify a cover?* -> Each edge has an endpoint in the set.
10. *Min VC of a triangle?* -> 2 (any two vertices).

7. Deep-Dive Questions

1. *Why is VC FPT but Clique/IndSet not?* -> The branching "each edge forces one of two endpoints" bounds the search tree by 2^k ; Clique has no such 2-way local forcing.
2. *Prove the 2-approx bound rigorously.* -> The picked edges form a matching M ; OPT must cover each edge in M with ≥ 1 vertex, so $\text{OPT} \geq |M|$; our cover = $2|M| \leq 2 \times \text{OPT}$.
3. *Can VC be approximated better than 2?* -> No known constant below 2; can't do better than ~ 1.36 unless $P=NP$; 2-eps ruled out under UGC.
4. *State and use Konig's theorem.* -> In bipartite graphs, max matching = min vertex cover, both polynomial.
5. *Kernelization for VC?* -> Reduce to a kernel of $O(k^2)$ (or $O(k)$) vertices before brute force; foundation of FPT practicality.

8. Comparison Table

	Vertex Cover	Independent Set	Dominating Set
Covers	every edge	nothing (no internal edges)	every vertex
Goal	minimize	maximize	minimize
Relation	$V \setminus \text{IndSet}$	$V \setminus \text{VC}$	different
Approx	2-approx	hard	log-approx
FPT in k	Yes	No	No

9. Common Mistakes

- "Covers every vertex" - it covers every **edge**.
- Confusing with Dominating Set.
- Thinking NP-complete means no good algorithm - VC has a 2-approx and FPT solution.

10. Edge Cases

- Graph with no edges: empty cover (size 0) suffices.
- Star graph: center alone covers everything.
- Complete graph K_n : $\min VC = n-1$.

11. How to Explain in Interview "A vertex cover is a set of vertices hitting every edge. Deciding a size- k cover is NP-complete, but it is unusually friendly: a trivial 2-approximation (take both ends of any uncovered edge) and an $O(2^k n)$ FPT algorithm make it tractable when the cover is small. Its complement is always an independent set."

12. Quick Revision

- VC = vertices touching every edge. Minimize.
- NP-complete, but 2-approximable and FPT in k .
- $\min VC + \max IS = |V|$. Bipartite: König (poly).
- Trap: covers edges, not vertices.

13. Practice Tasks

- Run the 2-approx on a 6-edge graph; compare to optimum.
- Implement the $O(2^k n)$ branching algorithm.
- Verify $\min VC + \max IS = |V|$ on a small graph.

14. Cheat Sheet

- **Def:** Fewest vertices covering every edge.
 - **Why:** NP-complete but 2-approx + FPT; monitoring/coverage.
 - **Most asked:** 2-approx proof; complement of IndSet; FPT.
 - **Compare:** VC vs IndSet vs Dominating Set.
 - **One-liner:** "Guard every edge with the fewest vertices - hard, but 2-approximable and FPT."
-

A6. Hamiltonian Cycle

1. Overview

- **Definition:** A **Hamiltonian cycle** visits **every vertex exactly once** and returns to the start. Decision: does graph G contain one?
- **Why it matters:** Foundational NP-complete problem; the backbone of TSP.
- **Where used:** Route/tour planning, DNA fragment assembly, circuit board drilling order, snake-like traversal problems.
- **Why interviewers ask:** The classic "why is this different from the easy Eulerian problem?" trap.

2. Core Idea

- **Intuition:** Find a single loop touching each vertex once. Verifying a proposed tour is trivial; finding one is exponential.
- **Analogy:** A delivery driver who must visit every house once and end back at the depot, never repeating a house.
- **Example:** Square A-B-C-D-A (4-cycle) is Hamiltonian. A graph missing edge D-A may not be.

- **Step-by-step verify:** given an ordering, check (a) it is a permutation of all vertices, (b) consecutive vertices (and last->first) are edges.

3. Important Subtopics

- **Hamiltonian Cycle vs Eulerian Circuit:** Hamiltonian = every **vertex** once (NP-complete); Eulerian = every **edge** once (P, just check even degrees + connectivity). *Angle:* THE classic trap.
- **Hamiltonian Path:** visits every vertex once but no return needed; also NP-complete.
- **Directed vs undirected** versions - both NP-complete.
- **Reduction to TSP:** assign edge weight 1 to existing edges, big weight otherwise; ask for a tour of total weight = $|V|$. *Angle:* "How does Hamiltonian relate to TSP?"

4. Real-World Example DNA sequencing (shortest superstring / fragment assembly) and PCB manufacturing (order to drill holes minimizing head travel) both reduce to Hamiltonian-path/TSP-style routing.

5. Mental Model

```
Eulerian (edges once)      -> EASY (P): all degrees even + connected
Hamiltonian (vertices once) -> HARD (NP-complete)
Hamiltonian Cycle --(unit weights)--> TSP
```

6. Common Interview Questions

1. *What is a Hamiltonian cycle?* -> Cycle visiting every vertex once. Mistake: saying every edge (that is Eulerian).
2. *Is it NP-complete?* -> Yes (both directed and undirected).
3. *Hamiltonian vs Eulerian?* -> Vertex-once (hard) vs edge-once (easy).
4. *How to check Eulerian quickly?* -> All vertices even degree + connected (circuit); exactly 0 or 2 odd for a path.
5. *Verify a Hamiltonian cycle?* -> Permutation of all vertices + consecutive edges exist.
6. *Hamiltonian path vs cycle?* -> Path no return; still NP-complete.
7. *Relate to TSP.* -> Unit weights present, large weights absent; tour = $|V|$ iff Hamiltonian.
8. *Is it in NP?* -> Yes, the tour ordering is the certificate.
9. *Complete graph K_n Hamiltonian?* -> Always ($n \geq 3$).
10. *Why no easy degree test like Euler?* -> No known local characterization; it is genuinely NP-complete.

7. Deep-Dive Questions

1. *Why is Eulerian easy but Hamiltonian hard?* -> Euler has a clean local characterization (degree parity); Hamiltonicity has no such local certificate - it is a global constraint with exponential search space.
2. *Sufficient conditions for Hamiltonicity?* -> Dirac (every vertex degree $\geq n/2$) and Ore ($\deg(u) + \deg(v) \geq n$ for non-adjacent u, v) guarantee a Hamiltonian cycle - but they are only sufficient, not necessary.
3. *Held-Karp DP complexity?* -> $O(2^n \cdot n^2)$ - exponential but far better than $n!$ brute force.
4. *Directed Hamiltonicity reduction?* -> Reduce from 3-SAT via chain/gadget constructions.
5. *Is Hamiltonicity poly on special graphs?* -> Yes for some (e.g., certain interval/tree-width-bounded classes).

8. Comparison Table

	Hamiltonian Cycle	Eulerian Circuit
Visits each...	vertex once	edge once
Complexity	NP-complete	P
Test	no simple test	all even degree + connected
Certificate	vertex ordering	edge ordering

9. Common Mistakes

- Mixing up Hamiltonian (vertices) with Eulerian (edges).
- Thinking a high-degree graph is automatically Hamiltonian (Dirac is sufficient, not necessary).
- Believing there is a simple degree test - there isn't.

10. Edge Cases

- Graph with a degree-1 vertex: no Hamiltonian cycle possible.
- Disconnected graph: none.
- Single vertex / two vertices: trivial or no cycle by convention.

11. How to Explain in Interview "A Hamiltonian cycle visits every vertex exactly once and loops back.

Deciding existence is NP-complete - unlike the Eulerian circuit (every edge once), which is polynomial via a simple degree-parity test. The contrast is the key insight: visiting edges once is local and easy; visiting vertices once is global and hard."

12. Quick Revision

- Hamiltonian = every vertex once, return to start. NP-complete.
- Contrast Eulerian (edges once) = P via degree test.
- Reduces to TSP with unit weights. Held-Karp DP = $O(2^n n^2)$.
- Dirac/Ore = sufficient conditions only.

13. Practice Tasks

- Decide Hamiltonicity of the Petersen graph (famously has a Hamiltonian *path* but no cycle).
- Implement Held-Karp bitmask DP.
- Check Eulerian vs Hamiltonian on the same small graph.

14. Cheat Sheet

- **Def:** Cycle through every vertex exactly once.
- **Why:** NP-complete; core of routing and TSP.
- **Most asked:** Hamiltonian vs Eulerian.
- **Compare:** vertex-once(hard) vs edge-once(easy).
- **One-liner:** "Visit every vertex once and return - hard, unlike the easy every-edge-once Eulerian tour."

A7. Traveling Salesman Problem (TSP)

1. Overview

- **Definition:** Given cities with pairwise distances, find the **shortest tour** visiting every city once and returning home. Decision form: is there a tour of length $\leq k$?
- **Why it matters:** The most famous NP-hard optimization problem; the "boss level" of routing.
- **Where used:** Logistics/delivery routing, vehicle fleet planning, manufacturing (drilling, welding order), genome assembly, circuit design.
- **Why interviewers ask:** Tests optimization vs decision, NP-hard vs NP-complete, and approximation algorithms.

2. Core Idea

- **Intuition:** A weighted Hamiltonian cycle where you also minimize total distance. Brute force is $(n-1)!/2$ tours - astronomically large.
- **Analogy:** A salesman planning the cheapest round-trip through all assigned cities.
- **Example:** 4 cities in a square, side 1, diagonal ~ 1.41 . Optimal tour = perimeter = 4, not crossing diagonals.
- **Step-by-step (decision verify):** given a tour, sum its edge weights, compare to k .

3. Important Subtopics

- **Decision (NP-complete) vs Optimization (NP-hard):** "tour $\leq k$?" is NP-complete; "shortest tour" is NP-hard (not in NP - can't verify optimality easily). *Angle:* THE NP-hard vs NP-complete example.
- **Metric TSP:** distances satisfy triangle inequality $\rightarrow$ Christofides gives a 1.5-approximation. *Angle:* "Can TSP be approximated?"
- **General TSP inapproximability:** without triangle inequality, no constant-factor approximation unless $P=NP$.
- **Exact methods:** Held-Karp DP $O(2^n n^2)$; branch-and-bound in practice.
- **Relation to Hamiltonian Cycle:** Hamiltonian Cycle reduces to TSP (unit/large weights).

4. Real-World Example Delivery companies (Amazon, UPS) solve TSP-like vehicle routing daily; they use heuristics (nearest neighbor, 2-opt, Lin-Kernighan, or Christofides-style) since exact optimal is infeasible at scale.

5. Mental Model

```
Hamiltonian Cycle (yes/no) --add weights + minimize--> TSP
Decision TSP ( $\leq k$ ) : NP-complete (verifiable)
Optimization TSP (min): NP-hard (optimality not poly-verifiable)
Metric TSP: Christofides 1.5-approx ; General: no constant approx.
```

6. Common Interview Questions

1. *What is TSP?* $\rightarrow$ Shortest tour visiting all cities once, return home. Mistake: forgetting the return.
2. *NP-hard or NP-complete?* $\rightarrow$ Optimization is NP-hard; decision ($\leq k$) is NP-complete.
3. *Why is optimization NP-hard but not NP-complete?* $\rightarrow$ Cannot verify a tour is *optimal* in poly time $\rightarrow$ not in NP.
4. *Relate TSP and Hamiltonian Cycle.* $\rightarrow$ HC reduces to TSP (unit weights, threshold $|V|$).
5. *Can TSP be approximated?* $\rightarrow$ Metric TSP: 1.5 (Christofides), 2 (MST doubling). General: no constant unless $P=NP$.

6. *Brute-force complexity?* -> $(n-1)!/2$ tours.
7. *Best exact DP?* -> Held-Karp, $O(2^n n^2)$.
8. *What is 2-opt?* -> Local search swapping two edges to shorten the tour; heuristic.
9. *Triangle inequality role?* -> Enables constant-factor approximations.
10. *Nearest-neighbor heuristic quality?* -> Fast but can be far from optimal; no good worst-case guarantee.

7. Deep-Dive Questions

1. *Why can't we verify optimality in poly time?* -> Verifying "no shorter tour exists" would require checking exponentially many alternatives; that is why optimization sits in NP-hard, not NP.
2. *Explain Christofides.* -> Build MST, find min-weight perfect matching on odd-degree vertices, combine into an Eulerian multigraph, shortcut into a Hamiltonian tour -> $\leq 1.5x$ optimal (metric).
3. *Why does MST-doubling give 2-approx?* -> Double MST edges (Eulerian), shortcut repeats via triangle inequality; $\text{tour} \leq 2 * \text{MST} \leq 2 * \text{OPT}$.
4. *General TSP hardness of approximation?* -> Any constant-factor approx would solve Hamiltonian Cycle, so it is NP-hard to approximate at all.
5. *TSP vs Vehicle Routing Problem?* -> VRP adds multiple vehicles, capacities, time windows - a harder generalization.

8. Comparison Table

	Decision TSP	Optimization TSP	Metric TSP
Question	tour $\leq k$?	shortest tour	shortest, triangle-ineq
Class	NP-complete	NP-hard	NP-hard
Verifiable?	yes	no (optimality)	no
Approx	-	none (general)	1.5 (Christofides)
	Hamiltonian Cycle	TSP	
Weights	none	yes	
Goal	existence	minimize length	
Class	NP-complete	NP-hard (opt)	

9. Common Mistakes

- Calling optimization TSP "NP-complete" - it is NP-hard (not in NP).
- Assuming any TSP is approximable - only the metric version is.
- Forgetting the "return to start" requirement.

10. Edge Cases

- 2 cities: trivial back-and-forth tour.
- Symmetric vs asymmetric TSP (distance $A \rightarrow B$ may differ from $B \rightarrow A$).
- Non-metric distances break Christofides.

11. How to Explain in Interview "TSP asks for the shortest round-trip visiting every city once. The decision version - is there a tour under length k - is NP-complete. The optimization version is NP-hard, and crucially

not in NP, because you cannot verify a tour is truly optimal in polynomial time. For metric distances Christofides gives a 1.5-approximation; general TSP has no constant-factor approximation unless $P=NP$."

12. Quick Revision

- TSP = shortest tour of all cities, return home.
- Decision: NP-complete; Optimization: NP-hard (not in NP).
- Metric: Christofides 1.5, MST-doubling 2. General: no constant approx.
- Held-Karp $O(2^n n^2)$; heuristics: NN, 2-opt, Lin-Kernighan.

13. Practice Tasks

- Solve a 5-city TSP by hand (enumerate tours).
- Implement nearest-neighbor + 2-opt improvement.
- Implement Held-Karp bitmask DP.

14. Cheat Sheet

- **Def:** Shortest tour visiting all cities once and returning.
- **Why:** Flagship NP-hard optimization; logistics.
- **Most asked:** NP-hard vs NP-complete; Christofides.
- **Compare:** decision(NP-complete) vs optimization(NP-hard); metric vs general.
- **One-liner:** "Weighted Hamiltonian cycle minimized - decision is NP-complete, optimization is the textbook NP-hard problem."

A8. Subset Sum

1. Overview

- **Definition:** Given a set of integers and a target T , is there a **subset that sums to exactly T** ?
- **Why it matters:** The simplest *number-theoretic* NP-complete problem; gateway to Partition and Knapsack.
- **Where used:** Budgeting/allocation, cryptography (knapsack cryptosystems), load balancing, coin/change decisions.
- **Why interviewers ask:** Perfect example of "NP-complete yet has a pseudo-polynomial DP" - the weak vs strong NP-completeness distinction.

2. Core Idea

- **Intuition:** 2^n possible subsets. But a DP over reachable sums runs in $O(n \cdot T)$ - polynomial in the *value* T but exponential in the number of *bits* of T .
- **Analogy:** Picking coins from a jar to hit an exact bill amount.
- **Example:** $\{3, 34, 4, 12, 5, 2\}$, $T=9 \rightarrow \{4, 5\} = 9 \rightarrow \text{YES}$. $T=30 \rightarrow \text{no subset} \rightarrow \text{NO}$.
- **Step-by-step (DP):** `reachable = {0}`; for each number x , `reachable |= (reachable + x)`; answer = T in `reachable`.

3. Important Subtopics

- **Pseudo-polynomial DP:** $O(nT)$ time. *Angle:* "Is Subset Sum in P?" $\rightarrow$ No; T can be exponential in input bit-length, so $O(nT)$ is not truly polynomial.

- **Weak NP-completeness:** hard only when numbers are huge; easy when values are small. *Angle:* the key nuance.
- **Meet-in-the-middle:** $O(2^{n/2})$ - better exact for large T . *Angle:* interview optimization.
- **3-SAT -> Subset Sum reduction:** encode variables/clauses as carefully constructed big numbers. *Angle:* proof of NP-completeness.

4. Real-World Example Cryptography: the Merkle-Hellman knapsack cryptosystem based its security on Subset Sum hardness (later broken, but historically important). Budget allocation ("can these expense items total exactly the budget?") is a direct instance.

5. Mental Model

Subsets: 2^n (exponential)
 DP over sums: $O(n \cdot T)$ (pseudo-polynomial - poly in value, not bit-length)
 Meet-in-middle: $O(2^{n/2})$

6. Common Interview Questions

1. *What is Subset Sum?* -> Subset summing exactly to T ? Mistake: " $\leq T$ " - it is exactly.
2. *Is it NP-complete?* -> Yes (weakly).
3. *Is it in P ?* -> No; the $O(nT)$ DP is pseudo-polynomial, not polynomial.
4. *What does pseudo-polynomial mean?* -> Poly in the numeric value T , exponential in T 's bit-length.
5. *DP recurrence?* -> $dp[i][s] = dp[i-1][s] \text{ OR } dp[i-1][s-a_i]$.
6. *Meet-in-the-middle idea?* -> Split into halves, enumerate each half's sums, match -> $O(2^{n/2})$.
7. *Relate to Partition.* -> Partition is Subset Sum with $T = \text{totalSum}/2$.
8. *Relate to Knapsack.* -> Knapsack generalizes it with values and a weight bound.
9. *Why weakly NP-complete?* -> Tractable when values are polynomially bounded.
10. *Empty subset?* -> Sums to 0; so $T=0$ is trivially YES.

7. Deep-Dive Questions

1. *Why isn't $O(nT)$ polynomial?* -> Input size is $O(n \log T)$ bits; T itself is exponential in $\log T$, so $O(nT)$ is exponential in input length.
2. *Sketch 3-SAT -> Subset Sum.* -> Construct base-large numbers with digit positions for each variable and clause so that a satisfying assignment forces digits to sum to a fixed target without carries.
3. *Strong vs weak NP-completeness?* -> Strong: hard even with unary/small numbers (e.g., 3-Partition); weak: only hard with large numbers (Subset Sum).
4. *FPTAS for Subset-Sum optimization?* -> Yes - trimming close sums yields a fully polynomial-time approximation scheme.
5. *Space optimization of the DP?* -> 1D boolean array of size $T+1$, iterate items, update sums downward.

8. Comparison Table

	Subset Sum	Partition	Knapsack
Target	exact T	$\text{totalSum}/2$	value $\geq V$ within weight W
Items have	value only	value only	weight + value

	Subset Sum	Partition	Knapsack
Class	weak NPC	weak NPC	weak NPC
DP	$O(nT)$	$O(n \cdot \text{sum})$	$O(nW)$

9. Common Mistakes

- Thinking the $O(nT)$ DP proves Subset Sum is in P.
- Reading "exactly T" as "at most T."
- Forgetting the empty subset gives sum 0.

10. Edge Cases

- $T=0$: always YES (empty set).
- Negative numbers: DP needs offset/shift.
- All numbers larger than T: only T reachable if some equals T.

11. How to Explain in Interview "Subset Sum asks whether some subset hits an exact target T. It is NP-complete, but only *weakly* - the $O(nT)$ DP solves it fast when numbers are small. It is not in P because T can be exponential in the input's bit-length. It is the parent of Partition and Knapsack."

12. Quick Revision

- Subset summing exactly to T. Weakly NP-complete.
- $O(nT)$ pseudo-polynomial DP; meet-in-middle $O(2^{n/2})$.
- Not in P: T exponential in bit-length.
- Parent of Partition ($T = \text{sum}/2$) and Knapsack.

13. Practice Tasks

- Implement the $O(nT)$ boolean DP.
- Implement meet-in-the-middle for $n=40$.
- Trace the DP table for $\{3,4,5,2\}$, $T=9$.

14. Cheat Sheet

- **Def:** Subset summing exactly to target T?
- **Why:** Simplest number NPC; weak NP-completeness demo.
- **Most asked:** pseudo-poly DP; why not in P.
- **Compare:** Subset Sum vs Partition vs Knapsack.
- **One-liner:** "Hit an exact target with a subset - NP-complete, but a pseudo-polynomial DP makes it easy for small numbers."

A9. Partition

1. Overview

- **Definition:** Given a set of positive integers, can it be **split into two subsets with equal sums**?
- **Why it matters:** Special case of Subset Sum (target = totalSum/2); models fair/balanced division.

- **Where used:** Load balancing across two machines, fair resource splitting, multiprocessor scheduling (2 processors), minimizing makespan.
- **Why interviewers ask:** Clean, intuitive NP-complete problem that connects directly to Subset Sum.

2. Core Idea

- **Intuition:** If total sum S is odd, immediately NO. Otherwise ask Subset Sum for target $S/2$.
- **Analogy:** Splitting a bill or a pile of weights into two equal halves.
- **Example:** $\{1,5,11,5\} \rightarrow S=22$, half=11 $\rightarrow \{11\}$ and $\{1,5,5\} \rightarrow \text{YES}$. $\{1,2,5\} \rightarrow S=8$, half=4 $\rightarrow$ no subset sums to 4 $\rightarrow$ NO.
- **Step-by-step:** compute S ; if odd $\rightarrow$ NO; else Subset-Sum DP for $S/2$.

3. Important Subtopics

- **Reduction from/to Subset Sum:** Partition = Subset Sum with $T = S/2$. *Angle:* "Relate Partition and Subset Sum."
- **Odd-sum shortcut:** parity check is an instant NO. *Angle:* edge case.
- **Balanced-partition optimization:** minimize the difference between the two subset sums (closest split); solved by the same DP. *Angle:* practical version.
- **3-Partition (different, strongly NP-complete):** split into triples of equal sum - do NOT confuse with Partition. *Angle:* trap.

4. Real-World Example Multiprocessor scheduling: distributing jobs across two identical machines to finish at the same time (minimize makespan) is the balanced-partition problem.

5. Mental Model

```
S = total sum
if S odd -> NO
else -> Subset Sum(target = S/2)
Balanced version: minimize | sumA - sumB |
```

6. Common Interview Questions

1. *What is Partition?* $\rightarrow$ Split into two equal-sum subsets. Mistake: allowing unequal or more than two parts.
2. *Is it NP-complete?* $\rightarrow$ Yes (weakly), via Subset Sum.
3. *Relate to Subset Sum.* $\rightarrow$ Target = totalSum/2.
4. *Quick NO condition?* $\rightarrow$ Odd total sum.
5. *DP complexity?* $\rightarrow O(n * S)$ pseudo-polynomial.
6. *Balanced partition?* $\rightarrow$ Minimize difference; same DP, pick reachable sum closest to $S/2$.
7. *Partition vs 3-Partition?* $\rightarrow$ 3-Partition (equal-sum triples) is *strongly* NP-complete - much harder, different problem.
8. *Is it in P?* $\rightarrow$ No; pseudo-polynomial only.
9. *Empty/single element?* $\rightarrow$ Single element (nonzero) $\rightarrow$ NO.
10. *Application?* $\rightarrow$ Two-machine load balancing.

7. Deep-Dive Questions

1. *Why weakly NP-complete?* -> Inherits Subset Sum's pseudo-polynomial DP; tractable for small sums.
2. *Why is 3-Partition strongly NP-complete but Partition only weakly?* -> 3-Partition stays hard even with numbers bounded by a polynomial (unary), so no pseudo-poly rescue.
3. *Approximation for balanced partition?* -> Karmarkar-Karp (largest differencing) heuristic gives good near-balanced splits.
4. *Space-optimized DP?* -> 1D boolean array of size $S/2$.
5. *Connection to makespan minimization?* -> Two-machine makespan = minimize the larger subset sum = balanced partition.

8. Comparison Table

	Partition	Subset Sum	3-Partition
Split into	2 equal halves	subset = T	triples, equal sum
Target	$S/2$	given T	$S/(n/3)$ each
NP-completeness	weak	weak	strong
Pseudo-poly DP	yes	yes	no help

9. Common Mistakes

- Confusing Partition (2 equal parts, weak) with 3-Partition (triples, strong).
- Forgetting the odd-sum instant NO.
- Assuming pseudo-poly DP means it is in P.

10. Edge Cases

- Odd total -> immediate NO.
- Empty set -> trivially YES (two empty halves, sum 0 each).
- Single nonzero element -> NO.

11. How to Explain in Interview "Partition asks whether a set splits into two equal-sum halves. It is exactly Subset Sum with target = total/2, so it is weakly NP-complete with the same $O(nS)$ DP. First check parity: an odd total is an instant no. Do not confuse it with 3-Partition, which is strongly NP-complete."

12. Quick Revision

- Split into two equal-sum subsets. Weakly NP-complete.
- = Subset Sum with $T = S/2$. Odd sum -> NO.
- $O(nS)$ DP. Balanced version minimizes the gap.
- 3-Partition is a different, strongly-NPC problem.

13. Practice Tasks

- Implement Partition via Subset-Sum DP.
- Implement balanced partition (min difference).
- Test $\{3, 1, 1, 2, 2, 1\}$ for an equal split.

14. Cheat Sheet

- **Def:** Split integers into two equal-sum subsets.

- **Why:** Fair division / load balancing; Subset Sum special case.
- **Most asked:** relation to Subset Sum; odd-sum shortcut; vs 3-Partition.
- **Compare:** Partition vs Subset Sum vs 3-Partition.
- **One-liner:** "Subset Sum with target half the total - weakly NP-complete, instant no if the total is odd."

A10. Knapsack (0/1)

1. Overview

- **Definition:** Given items with weights and values and a capacity W , choose items to **maximize value without exceeding weight W** (0/1 = take or leave each item). Decision: is value $\geq V$ achievable within W ?
- **Why it matters:** The canonical optimization-under-constraint problem; generalizes Subset Sum.
- **Where used:** Budget allocation, cargo/container loading, resource-constrained scheduling, portfolio selection, ad/keyword selection.
- **Why interviewers ask:** The most-asked DP problem *and* a weak NP-complete problem - tests both algorithm design and complexity nuance.

2. Core Idea

- **Intuition:** For each item decide take/skip; the DP tracks best value per capacity. $O(nW)$ - pseudo-polynomial.
- **Analogy:** A thief with a bag of capacity W picking the most valuable loot that fits.
- **Example:** capacity 5; items (w,v) : (2,3),(3,4),(4,5),(5,6). Best within 5 = items (2,3)+(3,4)=value 7.
- **Step-by-step (DP):** $dp[w] = \max(dp[w], dp[w - wt_i] + val_i)$ iterating items, capacity descending.

3. Important Subtopics

- **0/1 vs Fractional Knapsack:** 0/1 is NP-complete (DP); Fractional is in **P** via greedy by value/weight ratio. *Angle:* THE key contrast.
- **Pseudo-polynomial DP:** $O(nW)$, poly in W not its bit-length. *Angle:* weak NP-completeness.
- **Relation to Subset Sum:** set value = weight, $V = W \rightarrow$ Subset Sum. *Angle:* it is a generalization.
- **FPTAS:** scale values to get a $(1-\epsilon)$ -approximation in poly time. *Angle:* "approximate Knapsack well?"
- **Unbounded/bounded variants:** unlimited copies (unbounded) or limited counts.

4. Real-World Example Cloud cost optimization / cargo loading: choose which workloads or packages to include to maximize value/throughput under a fixed capacity or budget - a direct 0/1 Knapsack.

5. Mental Model

Fractional Knapsack: greedy by value/weight $\rightarrow$ P (optimal)
 0/1 Knapsack: DP $O(nW)$ $\rightarrow$ weakly NP-complete
 Subset Sum = Knapsack with value == weight, $V == W$
 FPTAS: scale values $\rightarrow$ $(1-\epsilon)$ approx in poly time

6. Common Interview Questions

1. *What is 0/1 Knapsack?* -> Maximize value within weight W, each item take/leave. Mistake: describing the fractional version.
2. *Is it NP-complete?* -> Decision form yes (weakly); optimization is NP-hard.
3. *0/1 vs Fractional?* -> Fractional is greedy/P; 0/1 needs DP and is NP-complete.
4. *Why is greedy wrong for 0/1?* -> Can't split items; a high-ratio item may waste capacity.
5. *DP recurrence?* -> $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt_i] + val_i)$.
6. *Complexity?* -> $O(nW)$ time, $O(W)$ space (1D).
7. *Is $O(nW)$ polynomial?* -> No - pseudo-polynomial (W exponential in bit-length).
8. *Relate to Subset Sum.* -> value=weight, $V=W$ special case.
9. *Can it be approximated?* -> Yes, FPTAS via value scaling.
10. *Unbounded Knapsack?* -> Unlimited copies; iterate capacity ascending.

7. Deep-Dive Questions

1. *Why does greedy work for Fractional but not 0/1?* -> Fractional lets you fill the last bit of capacity with a fraction of the best-ratio item (exchange argument holds); 0/1's indivisibility breaks the exchange argument.
2. *Explain the FPTAS.* -> Scale values by a factor based on eps, run the $O(n^2 * \maxval)$ value-based DP; rounding loses at most an eps fraction of optimal.
3. *Value-based DP alternative?* -> $dp[value] = \min \text{weight to reach it}$, $O(n * \text{sum of values})$ - useful when values are small.
4. *Why weakly NP-complete?* -> Pseudo-poly DP; hard only with large W or values.
5. *Bounded knapsack optimization?* -> Binary-splitting counts into powers of two to reduce to 0/1.

8. Comparison Table

	0/1 Knapsack	Fractional Knapsack	Subset Sum
Item split?	no	yes	no
Method	DP	greedy (ratio)	DP
Complexity	weak NPC	P	weak NPC
Optimal greedy?	no	yes	n/a

9. Common Mistakes

- Applying greedy to 0/1 Knapsack (only correct for fractional).
- Claiming $O(nW)$ proves it is in P.
- Confusing 0/1, unbounded, and bounded variants.

10. Edge Cases

- $W=0$: value 0 (nothing fits).
- Item heavier than W: never included.
- Zero-value items: skip; zero-weight positive-value items: always take.

11. How to Explain in Interview "0/1 Knapsack maximizes value under a weight cap with indivisible items. It is weakly NP-complete, solved by an $O(nW)$ DP - pseudo-polynomial, not truly polynomial. The fractional

version, where items are divisible, is in P via a simple greedy by value-to-weight ratio. Knapsack generalizes Subset Sum, and admits an FPTAS."

12. Quick Revision

- 0/1 Knapsack: maximize value within W , indivisible items. Weakly NP-complete.
- $O(nW)$ DP; $O(W)$ space. Not in P (pseudo-poly).
- Fractional = greedy = P. FPTAS exists.
- Generalizes Subset Sum (value=weight).

13. Practice Tasks

- Implement 0/1 Knapsack DP (2D then 1D).
- Implement Fractional Knapsack greedy; compare results.
- Reduce a Subset Sum instance to Knapsack and solve.

14. Cheat Sheet

- **Def:** Max value within weight W , take/leave items.
- **Why:** Constraint optimization; classic DP + weak NPC.
- **Most asked:** 0/1 vs fractional; why greedy fails for 0/1.
- **Compare:** 0/1(NPC/DP) vs Fractional(P/greedy) vs Subset Sum.
- **One-liner:** "Maximize loot value under a weight cap - weakly NP-complete via $O(nW)$ DP, unlike the greedy-solvable fractional version."

A11. Graph Coloring

1. Overview

- **Definition:** Assign colors to vertices so **no two adjacent vertices share a color**, using at most k colors (proper k -coloring). Decision: is G k -colorable? The **chromatic number** is the minimum such k .
- **Why it matters:** Models conflict-free assignment; 3-coloring is NP-complete, a workhorse reduction target.
- **Where used:** Register allocation in compilers, exam/timetable scheduling, frequency assignment (cell towers, Wi-Fi channels), Sudoku, map coloring.
- **Why interviewers ask:** Tests the 2 vs 3 complexity jump and real compiler applications.

2. Core Idea

- **Intuition:** Group vertices into color classes (each class is an independent set) so neighbors differ. Minimizing colors is the chromatic number.
- **Analogy:** Scheduling exams so no student has two exams at the same time - conflicting exams (edge) get different slots (colors).
- **Example:** A triangle needs 3 colors. A path or even cycle needs 2 (bipartite). An odd cycle needs 3.
- **Step-by-step verify:** given a coloring, check every edge has differently-colored endpoints.

3. Important Subtopics

- **2-coloring vs 3-coloring:** 2-coloring = bipartiteness check, in **P** (BFS/DFS). 3-coloring is NP-complete. *Angle:* THE complexity jump.

- **Chromatic number:** min colors; computing it is NP-hard.
- **Register allocation:** interference graph coloring with k = number of registers. *Angle:* the flagship real application.
- **Four Color Theorem:** any planar graph is 4-colorable (proven); yet deciding planar 3-colorability is still NP-complete. *Angle:* subtle trap.
- **Greedy coloring:** uses $\leq (\text{max degree} + 1)$ colors; not optimal. *Angle:* heuristic bound.

4. Real-World Example Compilers (LLVM/GCC): variables that are simultaneously live "interfere" (edge); coloring the interference graph with k physical registers assigns registers; if it needs more than k colors, some variables "spill" to memory.

5. Mental Model

```
2-coloring == bipartite check == P (BFS 2-color)
3-coloring == NP-complete
Each color class = an independent set
Chromatic number  $\chi(G)$  = min colors
Greedy uses  $\leq \text{maxdeg} + 1$  colors
```

6. Common Interview Questions

1. *What is graph coloring?* -> Color vertices so adjacent ones differ, minimize colors. Mistake: coloring edges (that is edge coloring).
2. *Is it NP-complete?* -> 3-coloring (and $k \geq 3$) yes; 2-coloring is P.
3. *2 vs 3 coloring complexity?* -> 2 = bipartite = P; 3 = NP-complete.
4. *How to check 2-colorability?* -> BFS/DFS assigning alternating colors; conflict -> not bipartite.
5. *What is the chromatic number?* -> Minimum colors for a proper coloring.
6. *Real application?* -> Register allocation; scheduling; frequency assignment.
7. *Four Color Theorem?* -> Planar graphs are 4-colorable; but planar 3-coloring is still NP-complete.
8. *Greedy coloring bound?* -> $\leq \text{maxdegree} + 1$ colors; order-dependent, not optimal.
9. *Chromatic number of a complete graph K_n ?* -> n .
10. *Odd cycle chromatic number?* -> 3; even cycle -> 2.

7. Deep-Dive Questions

1. *Why is 2-coloring easy but 3-coloring hard?* -> 2-coloring = bipartiteness, a locally-propagated constraint (BFS forces every color). With 3 colors each vertex has a real choice, creating exponential branching - reducible from 3-SAT.
2. *Sketch 3-SAT -> 3-Coloring.* -> Build a triangle of Base/True/False colors, variable gadgets (variable + negation), and clause OR-gadgets wired so a proper 3-coloring exists iff the formula is satisfiable.
3. *Chromatic number vs clique number?* -> $\chi(G) \geq \omega(G)$ (clique number); equality holds for perfect graphs but not in general.
4. *Register spilling connection?* -> If interference graph needs $> k$ colors, Chaitin's allocator spills a variable to memory and re-colors.
5. *Is 4-coloring planar graphs easy?* -> Existence is guaranteed (4CT), but finding a 3-coloring of a planar graph is still NP-complete.

8. Comparison Table

	2-Coloring	3-Coloring	k-Coloring ($k \geq 3$)
Meaning	bipartite?	3 colors?	k colors?
Complexity	P	NP-complete	NP-complete
Method	BFS/DFS	backtracking/SAT	backtracking/SAT
Vertex Coloring		Edge Coloring	
color vertices, neighbors differ		color edges, adjacent edges differ	
$\chi(G)$		$\chi'(G)$, Vizing: χ' in $\{D, D+1\}$	

9. Common Mistakes

- Thinking planar $\Rightarrow$ easy to 3-color (it is still NP-complete).
- Confusing vertex coloring with edge coloring.
- Assuming greedy gives the chromatic number (it does not).

10. Edge Cases

- No edges: 1 color suffices.
- Bipartite graph: exactly 2 colors (if it has an edge).
- Complete graph K_n : needs n colors.
- Odd cycle: 3 colors even though max degree is 2.

11. How to Explain in Interview "Graph coloring assigns colors to vertices so neighbors differ, minimizing colors (the chromatic number). Two-coloring is just a bipartiteness check - polynomial via BFS. Three-coloring is NP-complete, a common reduction target from 3-SAT. Its biggest real use is register allocation, where colors are CPU registers and conflicts force spills to memory."

12. Quick Revision

- Proper coloring: adjacent vertices differ. Minimize = chromatic number.
- 2-coloring = bipartite = P; 3-coloring = NP-complete.
- Each color class is an independent set.
- Uses: register allocation, scheduling, frequency assignment.
- 4CT: planar always 4-colorable; planar 3-coloring still NP-complete.

13. Practice Tasks

- Write a BFS bipartiteness (2-coloring) checker.
- Implement backtracking m-coloring.
- Model a small exam-scheduling conflict graph and color it.

14. Cheat Sheet

- **Def:** Color vertices, neighbors differ, min colors.
- **Why:** Conflict-free assignment; register allocation.
- **Most asked:** 2 vs 3 coloring jump; register allocation.
- **Compare:** 2-color(P) vs 3-color(NP); vertex vs edge coloring.

- **One-liner:** "Assign colors so no two neighbors clash - 2-coloring is easy (bipartite), 3-coloring is NP-complete."

PART B - CONCEPTUAL TOC QUESTIONS

These are the "explain the difference" and "why" questions that dominate TOC vivas and written exams. Each is answered to interview depth.

B1. DFA vs NFA

1. Overview

- **DFA (Deterministic Finite Automaton):** from each state, **exactly one** transition per input symbol; no epsilon moves. The machine's next state is always uniquely determined.
- **NFA (Nondeterministic Finite Automaton):** from a state, a symbol may lead to **zero, one, or many** states, and epsilon (empty-string) moves are allowed. The machine "guesses" / explores all paths in parallel.
- **Why it matters:** Both recognize *exactly* the regular languages; NFAs are easier to design, DFAs are easier to execute.
- **Why interviewers ask:** Tests the crucial idea that nondeterminism adds convenience, not power, for finite automata.

2. Core Idea

- **Intuition:** A DFA is a strict flowchart with one exit per input. An NFA can branch and accepts a string if **any** path reaches an accepting state.
- **Analogy:** DFA = a maze with one forced door per step. NFA = a maze where you can clone yourself at forks and win if any clone reaches the exit.
- **Example:** Language "strings ending in **ab**". NFA: stay in start on any symbol, then guess to move on **a** then **b**. DFA: must track the last-seen symbols explicitly.
- **Acceptance rule:** NFA accepts if at least one computation path ends in an accept state.

3. Important Subtopics

- **Subset construction (powerset):** converts any NFA to an equivalent DFA whose states are *sets* of NFA states. *Angle:* "Are they equivalent?" Yes, via this.
- **Epsilon transitions:** NFA-only convenience; removed via epsilon-closure. *Angle:* "Do epsilon moves add power?" No.
- **State blow-up:** an n -state NFA can become up to 2^n DFA states (worst case). *Angle:* the cost of determinizing.

4. Real-World Example Regex engines: a regular expression is compiled to an NFA (Thompson construction), then either simulated directly or converted to a DFA for fast, linear-time matching (used in **grep**, lexers like **flex**).

5. Mental Model

NFA (easy to design, branches) --subset construction--> DFA (easy to run, deterministic)
 Both recognize EXACTLY the regular languages.
 n-state NFA -> up to 2^n -state DFA.

6. Common Interview Questions

1. *DFA vs NFA?* -> Determinism of transitions; NFA allows multiple/epsilon moves. Mistake: claiming NFA is more powerful.
2. *Are they equivalent in power?* -> Yes, both = regular languages.
3. *How to convert NFA to DFA?* -> Subset construction (states = sets of NFA states).
4. *Worst-case DFA size?* -> Up to 2^n states.
5. *Do epsilon moves add power?* -> No; removed via epsilon-closure.
6. *Which is easier to design?* -> NFA. *To execute?* -> DFA.
7. *NFA acceptance condition?* -> Some path reaches accept.
8. *Space during NFA simulation?* -> Track a set of current states.
9. *Can every DFA be seen as an NFA?* -> Yes, DFA is a special case of NFA.
10. *Is minimization defined for DFA?* -> Yes (unique minimal DFA); NFAs have no unique minimal form.

7. Deep-Dive Questions

1. *Why is the DFA minimal form unique but NFA's not?* -> DFA minimization via Myhill-Nerode gives a canonical machine; NFA minimization is PSPACE-hard with no canonical form.
2. *Give a language with exponential DFA blow-up.* -> "nth symbol from the end is 1" needs 2^n DFA states but a small NFA.
3. *How does epsilon-closure work?* -> Set of states reachable via epsilon moves only; used in subset construction and simulation.
4. *NFA simulation time?* -> $O(\text{input length} * \text{states})$ tracking the active set - still linear in input.
5. *2DFA (two-way DFA) power?* -> Still only regular languages, despite moving the head both directions.

8. Comparison Table

Feature	DFA	NFA
Transitions per symbol	exactly 1	0, 1, or many
Epsilon moves	no	yes
Next state	unique	set of possibilities
Acceptance	end in accept state	any path accepts
Design ease	harder	easier
Execution	fast, direct	track state set
Power	regular languages	regular languages (same)
Minimal form	unique	not unique

9. Common Mistakes

- Saying NFA recognizes more languages than DFA.
- Forgetting epsilon-closure when converting.
- Thinking subset construction always causes 2^n blow-up (only worst case).

10. Edge Cases

- An NFA with a "dead" branch still accepts if another branch accepts.
- DFA must define transitions for every symbol (often via a dead/trap state).
- Empty language and empty-string acceptance handled by accept-state placement.

11. How to Explain in Interview "A DFA has exactly one transition per symbol; an NFA can branch to many states or take epsilon moves and accepts if any path reaches an accept state. They are equally powerful - both recognize exactly the regular languages - because subset construction turns any NFA into a DFA, at worst 2^n states. NFAs are easier to design; DFAs are easier to run."

12. Quick Revision

- DFA: deterministic, 1 transition/symbol. NFA: branching + epsilon.
- Equal power = regular languages. Subset construction converts NFA $\rightarrow$ DFA.
- Worst case 2^n blow-up. DFA has unique minimal form; NFA doesn't.

13. Practice Tasks

- Build an NFA for "ends in ab", convert to DFA via subset construction.
- Design a DFA for "even number of 1s".
- Convert an epsilon-NFA to a DFA by hand.

14. Cheat Sheet

- **Def:** DFA deterministic; NFA nondeterministic (branch/epsilon).
- **Why:** NFA convenient to design; DFA fast to run.
- **Most asked:** same power; subset construction; 2^n blow-up.
- **Compare:** transitions, epsilon, acceptance, minimal form.
- **One-liner:** "Same power, different convenience - any NFA becomes a DFA via subset construction."

B2. Why does non-determinism NOT make finite automata more powerful?

1. Overview

- **The claim:** NFAs and DFAs recognize *exactly the same class* of languages (regular languages). Nondeterminism buys smaller/simpler machines, not more expressive ones.
- **Why it matters:** It is the archetype of "nondeterminism = search convenience," and it contrasts sharply with the (open) P vs NP question where nondeterminism *might* matter.
- **Why interviewers ask:** It probes whether you understand *why*, via the subset construction proof, not just the fact.

2. Core Idea

- **Intuition:** An NFA in flight is really "in a *set* of possible states at once." That set is finite (at most 2^n subsets). A DFA can just make each such **set** one of its own states - so a DFA can simulate the NFA deterministically.
- **Analogy:** Tracking all positions a chess knight *could* be after k moves is itself a single, well-defined piece of information you can store and update deterministically.
- **Example:** NFA over $\{0,1\}$ with n states $\rightarrow$ DFA with states = subsets of those n states; the DFA tracks "which NFA states are currently active."
- **Step-by-step:** DFA start = epsilon-closure of NFA start; DFA transition on symbol a = epsilon-closure of the union of NFA moves; DFA accept = any subset containing an NFA accept state.

3. Important Subtopics

- **Finiteness is the crux:** only finitely many subsets of states exist, so the simulating DFA is still a *finite* automaton. *Angle:* "Why does the same trick fail to prove $P=NP$?" $\rightarrow$ a Turing machine's configuration set is *unbounded*, so you can't enumerate it into a finite deterministic structure.
- **Subset construction correctness:** DFA accepts w iff some NFA path on w accepts.
- **Cost:** convenience is paid in state count (up to 2^n), not in language power.

4. Real-World Example Lexical analyzers: tools like **flex** accept regex (naturally NFA-shaped), determinize to a DFA, then scan input in linear time - proving in practice that the nondeterministic spec and deterministic executor recognize the same tokens.

5. Mental Model

```
NFA "current state" = a SET of states (finite:  $\leq 2^n$  subsets)
DFA state := that set  $\rightarrow$  deterministic simulation
Finite state set  $\Rightarrow$  determinizable
(Turing machine config set is infinite  $\Rightarrow$  trick fails  $\Rightarrow$   $P$  vs  $NP$  open)
```

6. Common Interview Questions

1. Does nondeterminism add power to finite automata? $\rightarrow$ No. Mistake: "yes, NFAs accept more."
2. Why not? $\rightarrow$ Subset construction; finite subsets.
3. What is the DFA state in the simulation? $\rightarrow$ A set of NFA states.
4. What is the cost? $\rightarrow$ Up to 2^n states.
5. Why doesn't this argument settle P vs NP ? $\rightarrow$ TMs have infinitely many configurations; can't finitely determinize.
6. Do epsilon moves change this? $\rightarrow$ No, epsilon-closure handles them.
7. Is the conversion always possible? $\rightarrow$ Yes, for any NFA.
8. Does nondeterminism help PDAs? $\rightarrow$ Yes! NPDA $>$ DPDA (unlike finite automata) - key contrast.
9. So where does nondeterminism matter? $\rightarrow$ PDAs, and (conjecturally) Turing machines / P vs NP .
10. Is DFA size sometimes necessarily exponential? $\rightarrow$ Yes, some languages force 2^n .

7. Deep-Dive Questions

1. State the exact reason finiteness matters. $\rightarrow$ Determinization enumerates the reachable "belief states"; with finitely many states there are finitely many belief states, so the result is a finite machine.

2. *Contrast with PDAs.* -> A PDA's stack is unbounded, so its "belief state" isn't finite; nondeterministic PDAs are strictly stronger than deterministic ones.
3. *Contrast with Turing machines.* -> Determinizing an NTM by tracking all configurations needs unbounded memory and can blow up time exponentially - the crux of P vs NP.
4. *Is there a time cost to determinism for FAs?* -> Both run in linear time on input; only the machine size differs.
5. *Myhill-Nerode viewpoint?* -> The number of distinguishable prefixes (equivalence classes) is finite for a regular language, bounding both DFA and NFA expressiveness to the same class.

8. Comparison Table

Model	Nondeterminism adds power?	Why
Finite Automata	No	finite state set -> subset construction
Pushdown Automata	Yes (NPDA > DPDA)	unbounded stack, no finite determinization
Turing Machines	Open (P vs NP)	unbounded configurations

9. Common Mistakes

- Believing NFAs are strictly more powerful.
- Generalizing "nondeterminism = no extra power" to PDAs/TMs (false for PDAs, open for TMs).
- Forgetting that the equivalence costs exponential state blow-up.

10. Edge Cases

- Even with epsilon and heavy branching, the class stays regular.
- The blow-up is only worst-case; many NFAs determinize to small DFAs.

11. How to Explain in Interview "An NFA is always in a set of possible states, and since there are only finitely many such subsets - at most 2^n - a DFA can treat each subset as one of its own states and simulate the NFA deterministically. That is the subset construction. It works precisely because the state set is finite. The same idea fails for Turing machines, whose configurations are unbounded - which is exactly why P vs NP is still open."

12. Quick Revision

- NFA = DFA in power (regular languages), via subset construction.
- Works because subsets of a finite state set are finite ($\leq 2^n$).
- Cost: exponential state blow-up, not more languages.
- Contrast: nondeterminism DOES help PDAs; unknown for TMs (P vs NP).

13. Practice Tasks

- Determinize a 3-state NFA and count DFA states.
- Explain in writing why the argument breaks for PDAs.
- Find a language whose minimal DFA is exponentially larger than its NFA.

14. Cheat Sheet

- **Def:** NFA and DFA recognize the same regular languages.

- **Why:** Finite state set $\rightarrow$ subset construction determinizes.
- **Most asked:** the "set of states" argument; why it fails for TMs.
- **Compare:** FA(no gain) vs PDA(gain) vs TM(open).
- **One-liner:** "An NFA's live states form a finite set, so a DFA can track that set - nondeterminism only saves states, not language power."

B3. Regular Language vs Context-Free Language

1. Overview

- **Regular language (RL):** recognized by a finite automaton / described by a regular expression / generated by a regular grammar. **No memory** beyond finite state.
- **Context-free language (CFL):** generated by a context-free grammar / recognized by a pushdown automaton. Has **one stack** of memory - can count/match nested structure.
- **Why it matters:** Defines the Chomsky hierarchy boundary between "lexing" (regular) and "parsing" (context-free).
- **Why interviewers ask:** The $a^n b^n$ example and the pumping-lemma reasoning are TOC staples.

2. Core Idea

- **Intuition:** Regular = finite memory (can't count arbitrarily). Context-free = a stack, so it can match balanced/nested things (parentheses, $a^n b^n$).
- **Analogy:** Regular = a person who can only remember a fixed checklist. Context-free = a person with a stack of sticky notes (LIFO) to track nesting depth.
- **Example:** a^*b^* is regular. $a^n b^n$ (equal a's then b's) is **not** regular but **is** context-free. $a^n b^n c^n$ is not even context-free.
- **Step-by-step (why $a^n b^n$ isn't regular):** a DFA has finite states; it cannot remember an unbounded count of a's to match b's - pumping lemma formalizes this.

3. Important Subtopics

- **Containment:** every regular language is context-free (RL subset of CFL), not vice versa. *Angle:* "Is every regular language context-free?" Yes.
- **Pumping lemma (regular):** tool to *prove* a language is NOT regular. *Angle:* prove $a^n b^n$ non-regular.
- **Closure properties:** RLs are closed under intersection and complement; CFLs are **not** closed under intersection or complement. *Angle:* a favorite distinguishing fact.
- **Recognizer memory:** finite state (RL) vs one stack (CFL).

4. Real-World Example Compilers: the **lexer** uses regular languages (tokens like identifiers, numbers - regex). The **parser** uses context-free grammars (nested expressions, balanced braces, block structure) because regular languages can't match arbitrarily nested brackets.

5. Mental Model

Regular subset of Context-Free subset of Context-Sensitive subset of Recursively Enumerable
 Regular: finite memory (DFA/NFA, regex)

Context-Free: + one stack (PDA, CFG) -> handles $a^n b^n$, nesting
 $a^n b^n$: CFL not RL
 $a^n b^n c^n$: not even CFL

6. Common Interview Questions

1. *RL vs CFL?* -> Finite memory vs one stack. Mistake: saying CFL needs no automaton.
2. *Is every regular language context-free?* -> Yes; RL subset of CFL.
3. *Give a CFL that is not regular.* -> $a^n b^n$, balanced parentheses.
4. *How to prove a language non-regular?* -> Pumping lemma for regular languages.
5. *Which automaton for each?* -> Finite automaton (RL), pushdown automaton (CFL).
6. *Closure under intersection?* -> RL yes; CFL no.
7. *Closure under complement?* -> RL yes; CFL no (in general).
8. *Give a non-CFL language.* -> $a^n b^n c^n$, ww .
9. *Why can't RL do $a^n b^n$?* -> Finite states can't count unboundedly.
10. *Regex power = ?* -> Exactly regular languages (theoretical regex; real-world regex with backreferences exceeds this).

7. Deep-Dive Questions

1. *State the regular pumping lemma and use it.* -> For regular L there is p such that any string $|x| \geq p$ splits xyz with $|xy| \leq p$, $|y| \geq 1$, and $xy^i z$ in L for all i. For $a^n b^n$, pumping y (all a's) breaks the $a=b$ balance -> not regular.
2. *Why are CFLs not closed under intersection?* -> $a^n b^n c^m$ and $a^m b^n c^n$ are each CFL but their intersection is $a^n b^n c^n$, which is not CFL.
3. *Deterministic vs nondeterministic CFLs?* -> DCFL (LR-parsable) is a strict subset of CFL; nondeterminism adds power for PDAs.
4. *Is ww context-free?* -> No; but $w w^R$ (palindrome) is.
5. *Intersection of a CFL and a regular language?* -> Always context-free (useful closure property).

8. Comparison Table

Feature	Regular Language	Context-Free Language
Recognizer	Finite automaton	Pushdown automaton
Generator	Regular grammar / regex	Context-free grammar
Memory	finite state only	one stack
Example	a^*b^*	$a^n b^n$, balanced parens
Counting/nesting	no	yes (one level)
Closed under intersection	yes	no
Closed under complement	yes	no
Pumping lemma	regular version	CFL version

9. Common Mistakes

- Thinking $a^n b^n$ is regular (it is context-free only).
- Assuming CFLs are closed under intersection/complement (they are not).
- Believing real-world regex (with backreferences) equals theoretical regular languages.

10. Edge Cases

- Finite languages are always regular.
- $a^n b^m$ (independent counts) is regular; equality $a^n b^n$ is what breaks regularity.
- Regular intersect CFL is CFL (nice closure).

11. How to Explain in Interview "Regular languages are recognized with finite memory - finite automata or regex - so they can't count unboundedly. Context-free languages add a single stack, letting them match nested structure like $a^n b^n$ or balanced parentheses. Every regular language is context-free but not conversely. In compilers this is exactly the lexer (regular) versus parser (context-free) split."

12. Quick Revision

- RL: finite automaton/regex, finite memory. CFL: PDA/CFG, one stack.
- RL subset of CFL. $a^n b^n$ = CFL not RL; $a^n b^n c^n$ = not CFL.
- RL closed under intersection & complement; CFL is not.
- Pumping lemma proves non-membership.

13. Practice Tasks

- Prove $a^n b^n$ is not regular via the pumping lemma.
- Write a CFG for balanced parentheses.
- Build a DFA for a^*b^* and argue it can't do $a^n b^n$.

14. Cheat Sheet

- **Def:** RL = finite memory; CFL = one stack.
- **Why:** lexer vs parser; nesting needs a stack.
- **Most asked:** $a^n b^n$; closure differences.
- **Compare:** recognizer, memory, closure, examples.
- **One-liner:** "Add a stack to a finite automaton and regular becomes context-free - enough to match nesting like $a^n b^n$."

B4. CFG vs Regular Grammar

1. Overview

- **Regular grammar:** productions are restricted so all rules are **right-linear** ($A \rightarrow aB$ or $A \rightarrow a$) or all **left-linear**. Generates exactly the regular languages.
- **Context-free grammar (CFG):** productions have a **single nonterminal on the left**, any string of terminals/nonterminals on the right ($A \rightarrow \alpha$). Generates context-free languages.
- **Why it matters:** Grammar form is the "generator" side of the same regular vs context-free boundary.
- **Why interviewers ask:** Tests whether you can classify grammars by their production shape.

2. Core Idea

- **Intuition:** Regular grammars grow a string linearly in one direction (like a finite automaton walking). CFGs can expand a nonterminal into structure on both sides, enabling nesting and recursion ($A \rightarrow aAb$).
- **Analogy:** Regular grammar = laying bricks in a straight line. CFG = building a nested tree (parse tree) that can wrap content on both sides.
- **Example:** Regular: $S \rightarrow aS \mid b$ generates a^*b . CFG: $S \rightarrow aSb \mid \epsilon$ generates $a^n b^n$ (impossible for a regular grammar).
- **Step-by-step:** the presence of a rule like $A \rightarrow aAb$ (nonterminal flanked on both sides) makes it non-regular but context-free.

3. Important Subtopics

- **Right-linear vs left-linear:** a regular grammar must be consistently one or the other (mixing them can exceed regular). *Angle:* "What makes a grammar regular?"
- **Both sides recursion:** $A \rightarrow aAb \rightarrow$ context-free, not regular. *Angle:* the distinguishing rule shape.
- **Chomsky hierarchy:** Type 3 (regular) subset Type 2 (context-free). *Angle:* place them.
- **Every regular grammar is a CFG:** the converse fails.

4. Real-World Example Language spec docs: token grammars (identifiers, keywords) are given as regular grammars/regex; the language's syntax grammar (expressions, statements, nested blocks) is a CFG in BNF/EBNF fed to a parser generator like YACC/ANTLR.

5. Mental Model

Regular grammar: $A \rightarrow aB$ or $A \rightarrow a$ (nonterminal only at one end)
 CFG: $A \rightarrow$ any mix of terminals/nonterminals (e.g. aAb)
 Type 3 (regular) subset of Type 2 (context-free)

6. Common Interview Questions

1. *CFG vs regular grammar?* $\rightarrow$ Production shape: regular is linear (nonterminal at one end); CFG allows any right side. Mistake: ignoring the left-side single-nonterminal rule.
2. *What defines a regular grammar?* $\rightarrow$ All right-linear or all left-linear rules.
3. *Give a CFG that no regular grammar can produce.* $\rightarrow S \rightarrow aSb \mid \epsilon$ for $a^n b^n$.
4. *Is every regular grammar a CFG?* $\rightarrow$ Yes.
5. *Why is $A \rightarrow aAb$ not regular?* $\rightarrow$ Nonterminal flanked on both sides $\rightarrow$ needs a stack.
6. *Chomsky type numbers?* $\rightarrow$ Regular = Type 3, CFG = Type 2.
7. *Can mixing left- and right-linear rules stay regular?* $\rightarrow$ Not necessarily; consistency is required.
8. *What recognizes each?* $\rightarrow$ FA (regular grammar), PDA (CFG).
9. *Is $S \rightarrow aS \mid Sb \mid \epsilon$ regular?* $\rightarrow$ No (mixes directions / non-linear) - generates ab but the form isn't a proper single-direction regular grammar.
10. *Left side of a CFG rule?* $\rightarrow$ Exactly one nonterminal.

7. Deep-Dive Questions

1. *Why does both-sided recursion require a stack?* $\rightarrow$ Generating $a^n \dots b^n$ demands remembering how many a's were produced to emit matching b's - unbounded counting, needing a stack (PDA), which finite-state regular grammars lack.

2. Convert a right-linear grammar to an NFA. -> Nonterminals become states, $A \rightarrow aB$ becomes a transition $A \xrightarrow{a} B$, $A \rightarrow a$ goes to an accept state.
3. Are left-linear and right-linear grammars equally powerful? -> Yes, both generate exactly the regular languages.
4. Chomsky Normal Form? -> Every CFG converts to CNF ($A \rightarrow BC$ or $A \rightarrow a$), used by the CYK parser.
5. Ambiguity difference? -> Regular grammars can also be ambiguous, but ambiguity matters most for CFGs/parsing.

8. Comparison Table

Feature	Regular Grammar	Context-Free Grammar
Chomsky type	Type 3	Type 2
Rule form	$A \rightarrow aB$ / $A \rightarrow a$ (linear)	$A \rightarrow \alpha$ (any RHS)
Left side	one nonterminal	one nonterminal
Recursion	one-directional	both sides allowed
Generates	regular languages	context-free languages
Recognizer	finite automaton	pushdown automaton
Example	$S \rightarrow aS \mid b$	$S \rightarrow aSb \mid e$

9. Common Mistakes

- Forgetting a regular grammar must be consistently left- OR right-linear.
- Thinking any grammar with recursion is context-free-only (regular grammars have one-directional recursion too).
- Confusing the "single nonterminal on the left" rule (shared by both) with what distinguishes them (the right-hand side).

10. Edge Cases

- $A \rightarrow e$ (epsilon) allowed in both with care.
- A grammar can be context-free but still generate a regular language (form doesn't force the language class up).
- Linear grammars (nonterminal anywhere but at most one per RHS) sit between regular and general CFG.

11. How to Explain in Interview "Both have a single nonterminal on the left. A regular grammar restricts the right side to linear form - a nonterminal only at one end, like $A \rightarrow aB$ - so it maps directly to a finite automaton. A CFG allows any right-hand side, including both-sided recursion like $A \rightarrow aAb$, which lets it generate nested languages such as $a^n b^n$. Regular grammars are Type 3, CFGs Type 2, and every regular grammar is a CFG."

12. Quick Revision

- Regular grammar: right-/left-linear, Type 3, = regular languages.
- CFG: arbitrary RHS, Type 2, = context-free languages.
- $A \rightarrow aAb$ distinguishes CFG (needs a stack).

- Every regular grammar is a CFG, not vice versa.

13. Practice Tasks

- Convert $S \rightarrow aS \mid bS \mid e$ to an NFA.
- Write a CFG for balanced parentheses and one for palindromes.
- Classify given grammars as regular or context-free by rule form.

14. Cheat Sheet

- **Def:** Regular grammar = linear rules; CFG = any RHS.
- **Why:** token grammars vs syntax grammars.
- **Most asked:** rule shapes; $a^n b^n$ needs CFG.
- **Compare:** Type 3 vs Type 2; recognizer; recursion.
- **One-liner:** "A regular grammar keeps the nonterminal at one end; a CFG can wrap it on both sides, unlocking nesting."

B5. What makes a grammar ambiguous?

1. Overview

- **Definition:** A grammar is **ambiguous** if **some string** has **more than one distinct parse tree** (equivalently, more than one leftmost derivation).
- **Why it matters:** Ambiguity means a string has multiple *structural interpretations* - disastrous for compilers, where structure determines meaning (operator precedence, **if-else** binding).
- **Why interviewers ask:** The dangling-else and expression-grammar examples are classic; it tests understanding of derivations vs parse trees.

2. Core Idea

- **Intuition:** If the same sentence can be "diagrammed" two different ways, the grammar doesn't pin down meaning. $2 + 3 * 4$ could parse as $(2+3)*4$ or $2+(3*4)$ under a naive grammar.
- **Analogy:** The English sentence "I saw the man with the telescope" - two readings, two parse trees. Same words, different structure.
- **Example:** $E \rightarrow E + E \mid E * E \mid id$. The string $id + id * id$ has two parse trees (different operator groupings) $\rightarrow$ ambiguous.
- **Step-by-step to detect:** find one string with two distinct leftmost derivations / parse trees $\rightarrow$ proven ambiguous.

3. Important Subtopics

- **Parse tree vs derivation:** multiple *rightmost/leftmost* derivations that yield the *same* tree do NOT count; ambiguity needs different **trees**. *Angle:* common confusion.
- **Dangling else:** **if** E **then** **if** E **then** S **else** S - which **if** owns the **else**? Classic ambiguity. *Angle:* name it.
- **Disambiguation:** rewrite the grammar (precedence/associativity via layered nonterminals: $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow id$). *Angle:* "How to remove ambiguity?"
- **Inherent ambiguity:** some CFLs have **no** unambiguous grammar at all (e.g., $a^n b^n c^m$ union $a^n b^m c^m$). *Angle:* advanced trap.

- **Undecidability:** deciding whether an arbitrary CFG is ambiguous is **undecidable**. *Angle:* deep point.

4. Real-World Example Programming language parsers: expression grammars are carefully layered to encode precedence ($*$ binds tighter than $+$) and associativity; the dangling-else is resolved by the rule "else binds to the nearest unmatched if." Without this, $a - b - c$ could mean $a - (b - c)$.

5. Mental Model

```
Ambiguous <=> some string has >= 2 distinct parse trees
Fix: layer nonterminals to force precedence/associativity
E -> E + T | T
T -> T * F | F      (encodes * over +, left-associative)
F -> ( E ) | id
Ambiguity of an arbitrary CFG: UNDECIDABLE
```

6. Common Interview Questions

1. *What makes a grammar ambiguous?* -> A string with 2+ parse trees. Mistake: "2+ derivations" without specifying distinct trees.
2. *Give an ambiguous grammar.* -> $E \rightarrow E+E \mid E * E \mid id$.
3. *How to remove ambiguity?* -> Layer nonterminals for precedence/associativity.
4. *What is the dangling-else problem?* -> **else** can attach to two **ifs**; resolved by nearest-if rule.
5. *Parse tree vs derivation?* -> Ambiguity = multiple trees, not merely multiple derivations.
6. *Is ambiguity decidable?* -> No, undecidable for general CFGs.
7. *What is inherent ambiguity?* -> A language with no unambiguous grammar.
8. *Why does ambiguity matter in compilers?* -> Structure = semantics; ambiguity = undefined meaning.
9. *Can regular grammars be ambiguous?* -> Yes, but it's usually harmless (same string, one structure of meaning).
10. *Does ambiguity change the language?* -> No - same set of strings, different structures.

7. Deep-Dive Questions

1. *Why is ambiguity undecidable?* -> Reducible from the Post Correspondence Problem; no algorithm can decide it for all CFGs.
2. *Show the two parse trees for $id+id*id$.* -> One groups $(id+id)*id$, the other $id+(id*id)$; both derive the same string.
3. *What is an inherently ambiguous language?* -> $\{a^n b^n c^m d^m\} \cup \{a^n b^m c^m d^n\}$ -style languages have no unambiguous CFG.
4. *How do parser generators handle ambiguity?* -> Report shift/reduce or reduce/reduce conflicts (LR) and resolve via declared precedence.
5. *Is every LR(k) grammar unambiguous?* -> Yes; LR(k) grammars are unambiguous by construction (a plus of deterministic parsing).

8. Comparison Table

Aspect	Ambiguous grammar	Unambiguous grammar
Parse trees per string	≥ 2 for some string	exactly 1 for all

Aspect	Ambiguous grammar	Unambiguous grammar
Meaning	undefined/multiple	well-defined
Compiler use	problematic	required
Example	$E \rightarrow E+E \mid E^*E \mid id$	layered precedence grammar
Detection	undecidable in general	-
Confusion	Reality	
Multiple derivations = ambiguous	Only multiple <i>trees</i> count	
Ambiguous = different language	Same language, different structure	

9. Common Mistakes

- Equating "multiple leftmost/rightmost derivations" with ambiguity without checking the trees differ.
- Thinking ambiguity can always be removed (inherently ambiguous languages exist).
- Assuming a compiler can auto-detect ambiguity (undecidable).

10. Edge Cases

- A grammar can be ambiguous while the language it generates also has an unambiguous grammar.
- Some languages are inherently ambiguous - no fix exists.
- Left recursion is not itself ambiguity (but often removed for top-down parsing).

11. How to Explain in Interview "A grammar is ambiguous if some string has more than one distinct parse tree - meaning more than one structural interpretation. The classic examples are $E \rightarrow E+E \mid E^*E \mid id$, where $id+id*id$ parses two ways, and the dangling-else. We fix it by layering nonterminals to bake in precedence and associativity. Note that deciding ambiguity for an arbitrary CFG is undecidable, and some languages are inherently ambiguous."

12. Quick Revision

- Ambiguous = some string has ≥ 2 parse trees.
- Examples: $E \rightarrow E+E \mid E^*E \mid id$, dangling-else.
- Fix: layered grammar (precedence/associativity).
- Undecidable in general; inherent ambiguity exists; LR(k) $\Rightarrow$ unambiguous.

13. Practice Tasks

- Draw both parse trees for $id+id*id$.
- Rewrite $E \rightarrow E+E \mid E^*E \mid id$ into an unambiguous precedence grammar.
- Show the dangling-else ambiguity and its nearest-if resolution.

14. Cheat Sheet

- **Def:** Some string has multiple parse trees.
- **Why:** Structure = meaning; needed for correct compilers.
- **Most asked:** expression ambiguity, dangling-else, how to fix.
- **Compare:** ambiguous vs unambiguous; derivations vs trees.

- **One-liner:** "Two parse trees for one string - fix with precedence layering, but detecting it in general is undecidable."

B6. PDA vs Finite Automata

1. Overview

- **Finite Automaton (FA):** finite states, **no auxiliary memory**. Recognizes regular languages.
- **Pushdown Automaton (PDA):** an FA **plus a stack** (unbounded LIFO memory). Recognizes context-free languages.
- **Why it matters:** The stack is exactly what lets a PDA count/match nesting that an FA cannot.
- **Why interviewers ask:** Direct test of "what does adding a stack buy you," and the DPDA vs NPDA subtlety.

2. Core Idea

- **Intuition:** FA can only be in one of finitely many states. A PDA adds a stack, so it can remember an unbounded amount of *nested* information (push on **a**, pop on **b** to check $a^n b^n$).
- **Analogy:** FA = a person with a fixed mental checklist. PDA = same person holding a stack of plates - can track how deep the nesting goes.
- **Example:** To accept $a^n b^n$: push a symbol for each **a**, pop one for each **b**, accept if the stack empties exactly. An FA can't because it can't count unboundedly.
- **Acceptance:** by final state OR by empty stack (equivalent in power).

3. Important Subtopics

- **Stack = the difference:** unbounded LIFO memory. *Angle:* "Why is a PDA stronger than an FA?"
- **DPDA vs NPDA:** nondeterministic PDAs are **strictly more powerful** than deterministic ones (unlike FAs!). NPDA = all CFLs; DPDA = deterministic CFLs (a strict subset). *Angle:* THE contrast with finite automata.
- **Acceptance modes:** final state vs empty stack - equivalent for PDAs.
- **Limitation:** one stack only - can't do $a^n b^n c^n$ (would need two counters).

4. Real-World Example Parsers: a stack-based PDA models how a compiler parser tracks nested constructs - matching brackets, nested function calls, block scopes. Recursive-descent and LR parsers are essentially PDAs with an explicit stack.

5. Mental Model

```
FA:  states only           -> regular languages
PDA: states + one stack    -> context-free languages
      push on 'a', pop on 'b' -> recognizes  $a^n b^n$ 
NPDA (all CFLs) > DPDA (deterministic CFLs) <-- unlike FA where NFA = DFA
Two stacks == a Turing machine (too powerful)
```

6. Common Interview Questions

1. *PDA vs FA?* -> PDA adds a stack; recognizes CFLs vs regular. Mistake: forgetting the "one stack" limit.

2. What does the stack enable? -> Unbounded counting/nesting (e.g., $a^n b^n$).
3. Is a PDA more powerful than an FA? -> Yes, strictly (CFL superset of regular).
4. DPDA vs NPDA power? -> NPDA strictly stronger; DPDA = deterministic CFLs only.
5. Why is that different from NFA vs DFA? -> FAs: nondeterminism adds no power; PDAs: it does (unbounded stack can't be determinized).
6. Acceptance modes? -> Final state or empty stack, equivalent.
7. Can a PDA do $a^n b^n c^n$? -> No; one stack isn't enough.
8. What do two stacks give? -> Turing-machine power.
9. Example DCFL vs CFL? -> $a^n b^n$ is DCFL; ww^R (even palindromes) needs nondeterminism (CFL, not DCFL).
10. What recognizes DCFLs in practice? -> LR parsers.

7. Deep-Dive Questions

1. Why can't an NPDA be determinized like an NFA? -> The stack contents are unbounded, so the "set of configurations" isn't finite - subset construction fails.
2. Give a CFL that is not a DCFL. -> Even-length palindromes $w w^R$ - the machine must guess the midpoint (nondeterminism required).
3. Are DCFLs closed under complement? -> Yes (a nice property enabling deterministic parsing); general CFLs are not.
4. Why does two stacks equal a Turing machine? -> Two stacks simulate a two-way infinite tape (one stack = left of head, other = right).
5. Empty-stack vs final-state acceptance equivalence proof idea? -> Add a bottom marker and cleanup transitions to convert between the two modes.

8. Comparison Table

Feature	Finite Automaton	Pushdown Automaton
Memory	finite states	states + one stack
Language class	regular	context-free
Counting/nesting	no	yes (one level)
Nondeterminism adds power?	No (NFA=DFA)	Yes (NPDA>DPDA)
Example accepted	a^*b^*	$a^n b^n$
Real use	lexer	parser

9. Common Mistakes

- Treating DPDA and NPDA as equal in power (they are not - unlike DFA/NFA).
- Thinking a PDA can handle $a^n b^n c^n$ (needs more than one stack).
- Forgetting the two acceptance modes are equivalent.

10. Edge Cases

- A PDA with an unused stack is just an FA (regular).
- Empty stack at end vs final state - choose one acceptance mode consistently.
- Deterministic PDA needs a way to detect end-of-input for some languages.

11. How to Explain in Interview "A pushdown automaton is a finite automaton plus one unbounded stack. That stack lets it count and match nesting - push on each **a**, pop on each **b** to accept $a^n b^n$ - which a finite automaton cannot do. So PDAs recognize context-free languages, strictly more than regular. Crucially, unlike finite automata, nondeterministic PDAs are strictly more powerful than deterministic ones, because an unbounded stack can't be determinized by subset construction."

12. Quick Revision

- PDA = FA + one stack -> context-free languages.
- Stack enables $a^n b^n$, nesting, bracket matching.
- NPDA > DPDA (unlike NFA = DFA). Two stacks = Turing machine.
- One stack can't do $a^n b^n c^n$.

13. Practice Tasks

- Design a PDA for $a^n b^n$ (push/pop).
- Design a PDA for balanced parentheses.
- Argue why even palindromes need a nondeterministic PDA.

14. Cheat Sheet

- **Def:** PDA = finite automaton + one stack.
- **Why:** stack enables nesting/counting -> parsers.
- **Most asked:** what the stack buys; NPDA > DPDA.
- **Compare:** FA vs PDA memory, power, determinism.
- **One-liner:** "Add one unbounded stack to a finite automaton and you can match nesting - that's a PDA, and here nondeterminism genuinely adds power."

B7. Recursive vs Recursively Enumerable Language

1. Overview

- **Recursive (decidable) language:** a Turing machine **always halts** and correctly answers YES or NO for every input. Total decider.
- **Recursively enumerable (RE, recognizable) language:** a TM **halts and accepts** members, but on non-members it may **reject or loop forever**. Partial recognizer.
- **Why it matters:** The exact line between "solvable by algorithm" (recursive) and "only confirmable" (RE) - i.e., the limits of computation.
- **Why interviewers ask:** Tests understanding of halting behavior and where the halting problem sits.

2. Core Idea

- **Intuition:** Recursive = a machine that never gets stuck (always gives an answer). RE = a machine that says YES when true, but might run forever when the answer is NO.
- **Analogy:** Recursive = a search that always terminates with found/not-found. RE = a search that shouts "found!" if it exists, but on a bottomless list never confirms "not there."
- **Example:** "Does this program's syntax parse?" is recursive (decidable). "Does this TM halt on input w?" (halting problem) is RE but **not** recursive - you can confirm halting by simulating, but can't always confirm non-halting.

- **Step-by-step distinction:** recursive decides (always halts); RE recognizes (halts on YES, maybe loops on NO).

3. Important Subtopics

- **Containment:** Recursive subset of RE. Every decidable language is recognizable, not vice versa. *Angle:* place them.
- **Complement rule:** L is recursive **iff** both L and its complement are RE. *Angle:* a favorite theorem.
- **Co-RE:** complements of RE languages; halting problem's complement is co-RE but not RE. *Angle:* advanced.
- **Non-RE languages exist:** e.g., the complement of the halting problem. *Angle:* hierarchy top.

4. Real-World Example Program verification: "Will this program eventually print something?" is recognizable (run it - if it prints, you know) but not decidable (you can't always conclude it never will). Type checkers are decidable by design; general "does this loop terminate?" is not.

5. Mental Model

```
Recursive (decidable): TM always halts -> YES/NO
RE (recognizable):      TM halts on YES, may loop on NO
Recursive subset of RE subset of ALL languages
L recursive <=> L and complement(L) both RE
Halting problem: RE but NOT recursive
complement(Halting): NOT even RE
```

6. Common Interview Questions

1. *Recursive vs RE?* -> Always halts (decides) vs halts only on acceptance (recognizes). Mistake: swapping the two.
2. *Is every recursive language RE?* -> Yes; recursive subset of RE.
3. *Give an RE-but-not-recursive language.* -> The halting problem.
4. *When is a language recursive?* -> Iff both it and its complement are RE.
5. *What is co-RE?* -> Complements of RE languages.
6. *Is the complement of an RE language always RE?* -> No (else it'd be recursive).
7. *Do non-RE languages exist?* -> Yes (e.g., complement of halting).
8. *Recognizer behavior on non-members?* -> May reject or loop forever.
9. *Decider vs recognizer?* -> Decider always halts; recognizer may loop.
10. *Why care?* -> It marks the boundary of what algorithms can and cannot solve.

7. Deep-Dive Questions

1. *Prove: L recursive iff L and complement both RE.* -> Run both recognizers in parallel (dovetailing); one must halt-accept, telling you membership -> gives a total decider.
2. *Why is the halting problem RE but not recursive?* -> Simulate M on w: if it halts you accept (RE); but you can never be sure it won't halt later, so you can't decide NO -> not recursive.
3. *Where does the complement of halting sit?* -> co-RE, and provably not RE.
4. *Closure properties?* -> Recursive languages are closed under complement, union, intersection; RE closed under union/intersection but NOT complement.

5. *Rice's theorem connection?* -> Any nontrivial semantic property of TM languages is undecidable, populating the RE/non-recursive region.

8. Comparison Table

Feature	Recursive (Decidable)	Recursively Enumerable (Recognizable)
TM halts on...	all inputs	accepted inputs only
On non-members	halts, rejects	may reject OR loop
Also called	decidable	recognizable / Turing-acceptable
Complement	also recursive	not necessarily RE
Example	parsing, $a^n b^n c^n$ membership	halting problem
Closed under complement	yes	no

9. Common Mistakes

- Swapping the definitions (recursive = always halts).
- Thinking RE languages are closed under complement (they are not).
- Believing every language is at least RE (non-RE languages exist).

10. Edge Cases

- Finite languages and all regular/context-free languages are recursive.
- A language and its complement both RE forces recursive.
- The halting problem is the canonical RE-not-recursive example.

11. How to Explain in Interview "A recursive (decidable) language has a Turing machine that always halts with the right YES/NO. A recursively enumerable (recognizable) language has a machine that halts and accepts members but may loop forever on non-members. So recursive is a strict subset of RE. The key theorem: a language is recursive iff both it and its complement are RE. The halting problem is RE but not recursive - you can confirm halting, never guarantee non-halting."

12. Quick Revision

- Recursive: TM always halts (decides). RE: halts on YES, may loop on NO (recognizes).
- Recursive subset of RE. Recursive iff L and complement both RE.
- Recursive closed under complement; RE is not.
- Halting problem = RE not recursive; its complement is not even RE.

13. Practice Tasks

- List which language classes (regular, CFL, halting) are recursive vs RE.
- Explain the dovetailing decider for "L and complement both RE."
- Give one language in each region: recursive, RE-not-recursive, non-RE.

14. Cheat Sheet

- **Def:** Recursive = always halts; RE = halts on acceptance only.
- **Why:** boundary of algorithmic solvability.
- **Most asked:** containment; complement theorem; halting example.
- **Compare:** halting behavior, complement closure.
- **One-liner:** "Recursive machines always answer; RE machines confirm yes but may loop forever on no."

B8. Decidable vs Recognizable

1. Overview

- **Decidable:** same as recursive - a **decider** TM that always halts with YES/NO. The problem is *algorithmically solvable*.
- **Recognizable:** same as recursively enumerable - a **recognizer** TM that accepts (halts) on YES instances but may loop on NO instances.
- **Why it matters:** This is the same recursive/RE distinction phrased in the problem-solving language interviewers use ("Is X decidable?").
- **Why interviewers ask:** "Is the halting problem decidable?" is a near-guaranteed question.

2. Core Idea

- **Intuition:** Decidable = you get a guaranteed answer. Recognizable = you get a "yes" when it's yes, but possibly silence (infinite loop) when it's no.
- **Analogy:** Decidable = a vending machine that always returns your item or your money. Recognizable = a machine that dispenses if the item exists but may just hum forever if not.
- **Example:** "Is w in this regular language?" - decidable. "Does TM M accept w?" - recognizable but undecidable.
- **Step-by-step:** decidable => there is a total algorithm; recognizable => there is a semi-algorithm (may not terminate on no).

3. Important Subtopics

- **Decidable = Recognizable AND co-Recognizable:** if both a language and its complement are recognizable, it's decidable. *Angle:* the practical test.
- **Undecidable but recognizable:** the halting problem, **A_TM** (TM acceptance). *Angle:* name examples.
- **Undecidable and not recognizable:** **A_TM** complement, non-halting problem, equivalence of TMs. *Angle:* the non-RE region.
- **Reductions to prove undecidability:** reduce halting to a new problem. *Angle:* technique.

4. Real-World Example Static analysis tools: "Does this code have a type error?" is decidable (compilers do it). "Is this branch ever reachable / does this loop always terminate?" is undecidable in general - tools use conservative approximations (may report "unknown").

5. Mental Model

Decidable	= decider (always halts)	[= recursive]
Recognizable	= recognizer (halts on YES)	[= RE]
Decidable	<=> Recognizable AND co-Recognizable	

Undecidable examples: Halting, A_TM, Rice's theorem properties
 Not even recognizable: complement of Halting, TM equivalence

6. Common Interview Questions

1. *Decidable vs recognizable?* -> Always halts vs halts only on YES. Mistake: using them interchangeably.
2. *Is the halting problem decidable?* -> No; it is recognizable but undecidable.
3. *When is a problem decidable?* -> When it and its complement are both recognizable.
4. *Give an undecidable-but-recognizable problem.* -> A_TM (does M accept w?).
5. *Give a not-recognizable problem.* -> Complement of the halting problem; TM equivalence.
6. *What is Rice's theorem?* -> Every nontrivial semantic property of a program's language is undecidable.
7. *How to prove undecidability?* -> Reduce a known undecidable problem (halting) to it.
8. *Are decidable problems closed under complement?* -> Yes.
9. *Are recognizable problems closed under complement?* -> No.
10. *Decidable vs tractable?* -> Decidable = solvable at all; tractable = solvable in polynomial time (P).
Different axes.

7. Deep-Dive Questions

1. *Prove the halting problem is undecidable.* -> Diagonalization: assume decider H; build D that halts iff H says its input loops on itself; feed D to itself -> contradiction.
2. *State Rice's theorem precisely.* -> For any nontrivial property P of the language recognized by a TM, deciding whether a given TM's language has P is undecidable.
3. *Decidable vs recognizable vs co-recognizable Venn?* -> Decidable = intersection of recognizable and co-recognizable.
4. *Mapping reduction definition?* -> $A \leq_m B$ via computable f with $x \in A \iff f(x) \in B$; if B decidable then A decidable (used contrapositively for undecidability).
5. *Is equivalence of two CFGs decidable?* -> No, it is undecidable.

8. Comparison Table

Property	Decidable	Recognizable (RE)	Co-Recognizable (co-RE)
TM halts on YES	yes	yes	maybe loops
TM halts on NO	yes	maybe loops	yes
Also called	recursive	RE / Turing-acceptable	complement of RE
Example	regular membership	halting problem	non-halting problem
Decidable iff	-	+ co-recognizable	+ recognizable

9. Common Mistakes

- Treating "decidable" and "recognizable" as synonyms.
- Saying the halting problem is "unsolvable/uncomputable" without noting it is still recognizable.
- Confusing decidable (solvable at all) with tractable/P (solvable fast).

10. Edge Cases

- All regular and context-free membership problems are decidable.
- A problem can be recognizable yet its complement not - that forces undecidability.
- "Nontrivial" in Rice's theorem excludes always-true/always-false properties.

11. How to Explain in Interview "Decidable means a Turing machine always halts with the correct yes or no - the problem is algorithmically solvable. Recognizable (RE) means the machine halts and accepts on yes instances but may loop forever on no instances. A problem is decidable exactly when both it and its complement are recognizable. The halting problem is the textbook case: recognizable but undecidable, and its complement isn't even recognizable."

12. Quick Revision

- Decidable = decider always halts (recursive). Recognizable = halts on YES (RE).
- Decidable iff recognizable AND co-recognizable.
- Halting/A_TM: recognizable, undecidable. Their complements: not recognizable.
- Rice's theorem: nontrivial semantic properties are undecidable.
- Decidable != tractable (P is about *speed*).

13. Practice Tasks

- Prove the halting problem undecidable via diagonalization (write it out).
- Classify: TM emptiness, TM equivalence, CFG ambiguity - decidable or not.
- Show a mapping reduction from halting to **A_TM**.

14. Cheat Sheet

- **Def:** Decidable = always halts; Recognizable = halts on YES only.
- **Why:** limits of what algorithms can solve.
- **Most asked:** halting problem; decidable iff RE + co-RE.
- **Compare:** decidable vs RE vs co-RE.
- **One-liner:** "Decidable problems always answer; recognizable ones only confirm yes - the halting problem shows the gap is real."

B9. What is the Halting Problem?

1. Overview

- **Definition:** Given a program (Turing machine) M and input w, decide whether M **halts** (stops) on w or **runs forever**. Alan Turing (1936) proved **no algorithm can decide this for all (M, w)** - it is **undecidable**.
- **Why it matters:** The first and most famous undecidability result; it proves there are well-defined problems no computer can ever solve.
- **Why interviewers ask:** Cornerstone of computability; the diagonalization proof and its consequences are classic.

2. Core Idea

- **Intuition:** You cannot write one universal program that inspects any other program and always correctly says "this will finish" or "this will loop forever." Self-reference breaks any such claim.

- **Analogy:** A liar's paradox in code: build a program that does the opposite of what a halting-detector predicts about it - the detector must be wrong on that program.
- **Example:** `while(true){}` obviously loops; `print("hi")` obviously halts. But for arbitrary programs (e.g., "search for a counterexample to a conjecture") there's no general decider.
- **Step-by-step proof (diagonalization):**
 1. Assume a decider $H(M, w)$ returns HALT/LOOP correctly.
 2. Build $D(M)$: run $H(M, M)$; if H says "halts," loop forever; if "loops," halt.
 3. Run $D(D)$: if D halts, then H said it loops (so it shouldn't halt) - contradiction; if D loops, H said it halts - contradiction.
 4. So H cannot exist.

3. Important Subtopics

- **Undecidable but recognizable:** you can confirm halting (just simulate), never guarantee non-halting. *Angle:* place it in the RE hierarchy.
- **Diagonalization / self-reference:** the proof engine. *Angle:* explain the contradiction.
- **Reductions from halting:** prove other problems undecidable by reducing halting to them. *Angle:* technique.
- **Rice's theorem:** generalizes - all nontrivial semantic properties are undecidable. *Angle:* the bigger picture.

4. Real-World Example Why compilers/static analyzers can't perfectly detect infinite loops, dead code, or "will this ever throw": these reduce to the halting problem, so tools must be conservative (report "maybe") or bounded (timeouts). Antivirus can't perfectly decide if arbitrary code is malicious for the same reason.

5. Mental Model

```
Assume H decides halting.
Build D(M): if H(M,M)=halt -> loop; else -> halt.
Ask: does D(D) halt?
  D(D) halts => H(D,D)=loop => D(D) should loop (contradiction)
  D(D) loops => H(D,D)=halt => D(D) should halt (contradiction)
=> H cannot exist. Halting is UNDECIDABLE.
Status: RE (recognizable) but not recursive; complement not even RE.
```

6. Common Interview Questions

1. *What is the halting problem?* -> Decide if program M halts on input w . Mistake: "detect infinite loops sometimes" - it is the general case.
2. *Is it decidable?* -> No, proven undecidable by Turing.
3. *How is it proven?* -> Diagonalization / self-reference contradiction.
4. *Is it recognizable?* -> Yes (simulate to confirm halting), but not decidable.
5. *Why can't we just run the program and wait?* -> If it loops, you wait forever - you never conclude "loops."
6. *What's the practical consequence?* -> No perfect loop detector / static analyzer.
7. *How does it prove other things undecidable?* -> Via reductions from halting.
8. *Rice's theorem relation?* -> Generalizes halting: all nontrivial semantic properties undecidable.
9. *Does it mean computers are weak?* -> No; it's a fundamental limit, not an engineering flaw.

10. *Is its complement RE?* -> No - not even recognizable.

7. Deep-Dive Questions

1. *Walk through the full diagonalization.* -> (as in the mental model - construct D that contradicts H on input D).
2. *Why is simulation not a decider?* -> It semi-decides: halts-and-accepts on halting inputs, loops on non-halting ones.
3. *Reduce halting to program-equivalence.* -> Undecidability transfers, showing equivalence is undecidable too.
4. *What is the busy beaver connection?* -> Busy beaver is uncomputable precisely because halting is undecidable.
5. *Does bounded halting (halts within k steps?) become decidable?* -> Yes - just simulate k steps; the unboundedness is what makes it undecidable.

8. Comparison Table

Problem	Decidable?	Recognizable?
Halt within k steps	Yes	Yes
Halting (general)	No	Yes (RE)
Non-halting (complement)	No	No (not RE)
Program equivalence	No	No

Concept	Halting Problem
Discovered by	Turing (1936)
Proof technique	diagonalization
Class	RE but not recursive
Consequence	limits of static analysis

9. Common Mistakes

- Thinking it means "we just haven't found the algorithm yet" - it is provably impossible.
- Confusing bounded halting (decidable) with general halting (undecidable).
- Saying it's not even recognizable (it IS recognizable; its complement is not).

10. Edge Cases

- Specific programs may be analyzable; undecidability is about a *general* decider.
- Bounded/timeout versions are decidable.
- Total-language restrictions (e.g., primitive recursive) can make halting decidable but lose Turing-completeness.

11. How to Explain in Interview "The halting problem asks whether a given program halts on a given input. Turing proved it undecidable: assume a perfect halting detector H, then build a program D that loops when H predicts it halts and halts when H predicts it loops - running D on itself contradicts H, so H can't exist. It's

recognizable - you can confirm halting by simulation - but never decidable, because you can't confirm non-halting. This is why no tool can perfectly detect infinite loops."

12. Quick Revision

- Halting: does M halt on w? Undecidable (Turing, 1936).
- Proof: diagonalization / self-reference.
- RE (recognizable) but not recursive; complement not RE.
- Bounded halting is decidable. Basis for other undecidability via reduction.

13. Practice Tasks

- Write out the diagonalization proof in full.
- Explain why a timeout-based "loop detector" is only an approximation.
- Reduce halting to "does this program ever print?" to show the latter is undecidable.

14. Cheat Sheet

- **Def:** Decide if program M halts on input w.
- **Why:** first undecidability result; limits of computation.
- **Most asked:** the diagonalization proof; recognizable not decidable.
- **Compare:** bounded(decidable) vs general(undecidable); halting vs complement.
- **One-liner:** "No program can decide whether every other program halts - proven by a self-referential contradiction."

B10. P vs NP

1. Overview

- **P:** problems solvable in **polynomial time** (n^c) - "efficiently solvable."
- **NP:** problems whose YES-answers are **verifiable in polynomial time** given a certificate - "efficiently checkable."
- **P vs NP question:** Is $P = NP$? I.e., is everything checkable-fast also solvable-fast? Widely believed $P \neq NP$, but unproven (a \$1M Millennium Prize problem).
- **Why it matters:** It underpins cryptography, optimization, and the very notion of computational hardness.
- **Why interviewers ask:** The single most famous open problem in CS; tests the verify-vs-solve intuition.

2. Core Idea

- **Intuition:** P = "I can find the answer fast." NP = "If you show me an answer, I can check it fast." P vs NP asks whether checking-fast implies finding-fast.
- **Analogy:** A jigsaw puzzle - verifying a completed puzzle is instant (NP), but solving from scratch may be slow. Or: guessing a password is hard, checking one is easy.
- **Example:** Sudoku - verifying a filled grid is polynomial; solving a large one may not be. Factoring, SAT, TSP-decision are in NP .
- **Step-by-step (NP membership):** exhibit a polynomial-size certificate and a polynomial-time verifier.

3. Important Subtopics

- **P subset of NP:** if you can solve fast, you can verify fast (ignore the certificate). *Angle:* "Is P inside NP?" Yes.
- **The open question:** no proof either way; most believe $P \neq NP$. *Angle:* don't claim it's solved.
- **NP-complete role:** if ANY NP-complete problem is in P, then $P = NP$. *Angle:* why they matter.
- **co-NP:** problems whose NO-answers are verifiable; P vs NP vs co-NP relationships. *Angle:* advanced.
- **Consequences:** $P = NP$ would break most public-key crypto and trivialize optimization. *Angle:* stakes.

4. Real-World Example Cryptography: RSA relies on factoring being hard (in NP, not known to be in P). If $P = NP$, an efficient algorithm for NP-complete problems would likely break encryption, cracking passwords and secure communications everywhere.

5. Mental Model

```
P   = solvable in poly time      (find fast)
NP  = verifiable in poly time    (check fast)
P subset of NP (solving implies verifying)
P = NP ? -> OPEN. Believed NO.
If one NP-complete problem in P => P = NP (all of them collapse)
```

6. Common Interview Questions

1. *What is P?* -> Poly-time solvable. Mistake: "polynomial space."
2. *What is NP?* -> Poly-time verifiable with a certificate. Mistake: "not polynomial."
3. *Is P a subset of NP?* -> Yes.
4. *Is P = NP?* -> Unknown; believed no.
5. *What does NP stand for?* -> Nondeterministic Polynomial (poly time on a nondeterministic TM). Mistake: "Non-Polynomial."
6. *Give an NP problem.* -> SAT, clique, TSP-decision, subset sum.
7. *What if P = NP?* -> Efficient algorithms for all NP problems; crypto breaks.
8. *NP vs exponential problems?* -> NP problems have poly-time verifiers; some harder problems (EXPTIME) don't.
9. *What is co-NP?* -> NO-answers verifiable; e.g., tautology checking.
10. *How do NP-complete problems relate?* -> Hardest in NP; one in P collapses the whole class.

7. Deep-Dive Questions

1. *Two equivalent definitions of NP?* -> (a) verifiable in poly time by a deterministic verifier with a certificate; (b) decidable in poly time by a nondeterministic TM. They coincide.
2. *Why does P subset of NP hold?* -> A poly-time solver ignores the certificate and just solves; that's a valid verifier.
3. *Is NP closed under complement?* -> Unknown; whether $NP = co-NP$ is open (and $NP = co-NP$ would be a big deal).
4. *What is the significance of NP-completeness for P vs NP?* -> They are the "if any is easy, all are easy" anchors; resolving one resolves the class.
5. *Where does factoring sit?* -> In NP and co-NP, not known NP-complete; suspected intermediate (NP-intermediate, per Ladner's theorem which guarantees such problems exist if $P \neq NP$).

8. Comparison Table

Class	Definition	Example
P	poly-time solvable	sorting, shortest path, 2-SAT
NP	poly-time verifiable	SAT, clique, TSP-decision
co-NP	NO-answers verifiable	tautology, no-clique
NP-complete	hardest in NP	SAT, 3-SAT, clique
NP-hard	at least as hard as NP	TSP-optimization, halting

	Solve fast	Verify fast
P	yes	yes
NP	maybe (open)	yes

9. Common Mistakes

- "NP = Non-Polynomial" - it's Nondeterministic Polynomial.
- Claiming $P \neq NP$ is proven (it is conjectured).
- Thinking all NP problems are NP-complete (P problems are in NP too).
- Assuming NP means "hard" - P subset of NP, so easy problems are in NP.

10. Edge Cases

- Every problem in P is trivially in NP.
- NP-intermediate problems (factoring, graph isomorphism) are believed neither in P nor NP-complete.
- Decision vs optimization: NP is about decision problems.

11. How to Explain in Interview "P is problems solvable in polynomial time; NP is problems whose solutions can be *verified* in polynomial time given a certificate. Clearly P is inside NP - solving lets you verify. The P vs NP question asks whether verifying-fast implies solving-fast. It's open and widely believed false. It matters because if $P = NP$, every NP-complete problem - SAT, TSP, scheduling - becomes efficiently solvable, and modern cryptography collapses."

12. Quick Revision

- $P =$ solve fast; $NP =$ verify fast (certificate). P subset of NP .
- $NP =$ Nondeterministic Polynomial. P vs NP open; believed $P \neq NP$.
- One NP-complete problem in $P \Rightarrow P = NP$.
- $P = NP$ would break crypto. co-NP, NP-intermediate exist.

13. Practice Tasks

- For SAT, clique, subset sum: state the certificate and verifier.
- Argue why P subset of NP in two sentences.
- List 3 problems in P and 3 NP-complete problems.

14. Cheat Sheet

- **Def:** $P =$ poly-time solvable; $NP =$ poly-time verifiable.
- **Why:** defines efficient computation; underpins crypto.

- **Most asked:** verify vs solve; NP = Nondeterministic Poly; is $P=NP$ (open).
- **Compare:** P vs NP vs co-NP vs NP-complete.
- **One-liner:** "P is solve-fast, NP is check-fast; whether they're equal is computer science's biggest open question."

B11. NP-hard vs NP-complete

1. Overview

- **NP-hard:** "at least as hard as every problem in NP." Every NP problem reduces to it. It **may or may not be in NP** (could be even harder, or undecidable).
- **NP-complete: in NP AND NP-hard.** The hardest problems that are *still verifiable* in polynomial time.
- **Why it matters:** Precisely classifies "hard" problems and separates the verifiable-hard (NP-complete) from the possibly-worse (NP-hard).
- **Why interviewers ask:** The TSP example (optimization NP-hard, decision NP-complete) is a favorite discriminator.

2. Core Idea

- **Intuition:** NP-complete = hard *and* checkable. NP-hard = hard, but maybe you can't even verify a solution quickly (so not in NP). All NP-complete problems are NP-hard; not all NP-hard problems are NP-complete.
- **Analogy:** NP-complete = the hardest problems in a "verifiable club." NP-hard = anything at least that hard, possibly outside the club entirely.
- **Example:** SAT, 3-SAT, clique $\rightarrow$ NP-complete. TSP-optimization, halting problem $\rightarrow$ NP-hard but not NP-complete (halting isn't even decidable; TSP-optimization isn't in NP).
- **Step-by-step to show NP-complete:** (1) prove it's in NP (poly verifier), (2) reduce a known NP-complete problem to it (NP-hardness).

3. Important Subtopics

- **The two conditions:** NP-complete = NP membership + NP-hardness. *Angle:* "How to prove NP-completeness?"
- **NP-hard beyond NP:** includes undecidable problems (halting is NP-hard). *Angle:* the trap that NP-hard isn't always in NP.
- **Optimization vs decision:** optimization forms are often NP-hard, their decision forms NP-complete. *Angle:* TSP.
- **Cook-Levin:** SAT is the first NP-complete, giving a seed for all reductions. *Angle:* origin.

4. Real-World Example Route optimization (TSP-optimization) is NP-hard - companies use heuristics/approximations because exact optimal is infeasible. The *decision* version ("is there a route under X km?") is NP-complete. Recognizing that a scheduling problem is NP-complete tells engineers to stop seeking an exact poly algorithm and use approximation/heuristics.

5. Mental Model

NP-hard: every NP problem reduces to it ($\geq$ hardest NP)
 NP-complete = NP-hard AND in NP

[NP-complete] subset of [NP-hard]

Examples:

NP-complete: SAT, 3-SAT, clique, vertex cover, Hamiltonian cycle

NP-hard only: TSP-optimization, halting problem (not in NP)

Prove NP-complete: (1) in NP, (2) reduce known NPC problem to it.

6. Common Interview Questions

1. *NP-hard vs NP-complete?* -> NP-complete is in NP; NP-hard need not be. Mistake: treating them as identical.
2. *Is every NP-complete problem NP-hard?* -> Yes.
3. *Is every NP-hard problem NP-complete?* -> No (may not be in NP).
4. *Give an NP-hard but not NP-complete problem.* -> Halting problem, TSP-optimization.
5. *How to prove a problem NP-complete?* -> Show in NP + reduce a known NPC problem to it.
6. *Why is the halting problem NP-hard?* -> Every NP problem reduces to it, but it's undecidable, so not in NP.
7. *TSP decision vs optimization?* -> Decision NP-complete; optimization NP-hard.
8. *What's the first NP-complete problem?* -> SAT (Cook-Levin).
9. *Can an NP-hard problem be easy to verify?* -> Not necessarily; that's why it may be outside NP.
10. *If an NP-hard problem is solved in poly time, does $P=NP$?* -> Only if it's also in NP (i.e., NP-complete); a non-NP NP-hard problem in P still implies $P=NP$ by reduction, but such problems (like halting) can't be in P anyway.

7. Deep-Dive Questions

1. *Precise definitions?* -> B is NP-hard if for all A in NP, $A \leq_p B$. B is NP-complete if B in NP and B is NP-hard.
2. *Why isn't TSP-optimization in NP?* -> Verifying a tour is *optimal* isn't known to be poly-checkable (you'd need to rule out all shorter tours).
3. *Reduction direction for proofs?* -> To prove B NP-hard, reduce a known NP-hard/complete A **to** B ($A \leq_p B$), not the reverse.
4. *Are there NP-intermediate problems?* -> If $P \neq NP$, Ladner's theorem guarantees problems in NP that are neither in P nor NP-complete (e.g., suspected: factoring, graph isomorphism).
5. *NP-hard optimization vs NP-hard decision?* -> Optimization is at least as hard as its decision version; both NP-hard, but only the decision one can be NP-complete.

8. Comparison Table

Feature	NP-hard	NP-complete
In NP?	not necessarily	yes
NP-hard?	yes	yes
Verifiable in poly time?	maybe not	yes
Can be undecidable?	yes (e.g. halting)	no
Example	TSP-opt, halting	SAT, 3-SAT, clique
Proof needs	reduction from NPC	in NP + reduction

9. Common Mistakes

- Saying NP-hard implies in NP (it does not).
- Calling TSP-optimization NP-complete (it's NP-hard, not in NP).
- Reducing in the wrong direction when proving hardness.
- Thinking "NP-hard" means "definitely harder than NP" - it means "at least as hard."

10. Edge Cases

- The halting problem is NP-hard yet undecidable - the extreme example of "NP-hard but not in NP."
- Some NP-hard problems are also in NP (those are exactly the NP-complete ones).
- Optimization variants blur the line - always specify decision vs optimization.

11. How to Explain in Interview "NP-hard means at least as hard as every NP problem - everything in NP reduces to it - but it might not be in NP itself, so its solutions might not even be poly-verifiable; the halting problem is NP-hard yet undecidable. NP-complete is the intersection: NP-hard *and* in NP. To prove a problem NP-complete you show it's in NP and reduce a known NP-complete problem to it. TSP's decision version is NP-complete; its optimization version is NP-hard but not in NP."

12. Quick Revision

- NP-hard: everything in NP reduces to it; may be outside NP.
- NP-complete = NP-hard + in NP.
- Prove NPC: (1) in NP, (2) reduce known NPC to it.
- Halting = NP-hard not NP-complete. TSP-opt NP-hard, TSP-decision NP-complete.

13. Practice Tasks

- Classify: SAT, halting, TSP-opt, TSP-decision, clique as NP-hard/NP-complete.
- Write the two-step NP-completeness proof outline for clique.
- Explain why halting is NP-hard but not NP-complete.

14. Cheat Sheet

- **Def:** NP-hard = $\geq$ all NP; NP-complete = NP-hard AND in NP.
- **Why:** precise map of hardest verifiable problems.
- **Most asked:** difference; halting/TSP examples; how to prove NPC.
- **Compare:** NP-hard vs NP-complete (in-NP is the key axis).
- **One-liner:** "NP-complete is the hardest *checkable* problem; NP-hard is at least that hard but may not even be verifiable."

B12. How do Polynomial Reductions Work?

1. Overview

- **Definition:** A **polynomial-time reduction** $A \leq_p B$ is a polynomial-time computable function f that transforms any instance x of A into an instance $f(x)$ of B such that x is a YES for A iff $f(x)$ is a YES for B .
- **Why it matters:** Reductions are the *tool* that spreads hardness (and easiness) across problems - the entire NP-completeness theory is built on them.

- **Why interviewers ask:** Understanding reduction direction is the #1 source of confusion; they test it directly.

2. Core Idea

- **Intuition:** "If I can transform A into B cheaply, then B is at least as hard as A." Solving B lets me solve A (via f). So hardness flows from A to B.
- **Analogy:** Converting a problem in feet to one in meters using a fixed formula - if you can solve the metric version, you've solved the imperial one.
- **Example:** 3-SAT $\leq_p$ Clique. Given a 3-SAT formula, build a graph (vertices = literals, edges = compatible literals in different clauses) so that a clique of size (#clauses) exists iff the formula is satisfiable.
- **Step-by-step:** (1) define f mapping A-instances to B-instances, (2) prove f runs in poly time, (3) prove $\text{YES}(A) \iff \text{YES}(B)$.

3. Important Subtopics

- **Direction is everything:** to prove **B is hard**, reduce a **known-hard A to B** ($A \leq_p B$). Reducing B to A proves nothing about B's hardness. *Angle:* THE classic mistake.
- **Mapping (many-one) vs Turing reductions:** mapping = single transformation + preserve answer; Turing = use B as a subroutine multiple times. *Angle:* types.
- **Transitivity:** $A \leq_p B$ and $B \leq_p C$ implies $A \leq_p C$ - lets hardness chain. *Angle:* why SAT seeds everything.
- **Gadgets:** local structures encoding variables/clauses in the target problem. *Angle:* how graph reductions are built.

4. Real-World Example When an engineer proves a new scheduling problem is NP-complete, they don't start from scratch - they reduce a known NP-complete problem (like 3-SAT or Partition) to it, inheriting its hardness. This is exactly how thousands of problems were shown NP-complete after Cook-Levin.

5. Mental Model

$A \leq_p B$ means: transform A-instances into B-instances in poly time, preserving YES/NO.

Consequences:

B easy (in P) $\Rightarrow$ A easy (solve A via f then B)

A hard $\Rightarrow$ B hard (else A would be easy)

To prove B NP-hard: pick known NP-complete A, show $A \leq_p B$.

Transitive: $A \leq_p B \leq_p C \Rightarrow A \leq_p C$.

6. Common Interview Questions

1. *What is a poly reduction?* -> Poly-time map preserving YES/NO from A to B. Mistake: forgetting the "iff" answer-preservation.
2. *Which direction to prove B is NP-hard?* -> Reduce known-hard A **to** B ($A \leq_p B$).
3. *What does $A \leq_p B$ tell you?* -> B is at least as hard as A.
4. *If B is in P and $A \leq_p B$?* -> A is in P too.
5. *Reduction time bound?* -> Polynomial.

6. *Mapping vs Turing reduction?* -> Single transform preserving answer vs using B as a subroutine.
7. *Why must it be poly time?* -> So it doesn't add exponential cost that hides the real hardness.
8. *Is $\leq_p$ transitive?* -> Yes.
9. *Give a reduction example.* -> 3-SAT $\leq_p$ Clique (gadget construction).
10. *What is a gadget?* -> A local structure encoding part of A inside B.

7. Deep-Dive Questions

1. *Prove correctness of a reduction generally.* -> Show both directions: YES(x) in A $\Rightarrow$ f(x) YES in B, and f(x) YES in B $\Rightarrow$ x YES in A (no false positives/negatives).
2. *Why does the wrong direction prove nothing?* -> Reducing B to an easy A only shows B is no harder than A; it says nothing about a lower bound on B.
3. *How does Cook-Levin bootstrap the theory?* -> It reduces *every* NP problem to SAT directly; then transitivity spreads NP-hardness from SAT to any problem SAT reduces to.
4. *Turing reduction subtlety for NP?* -> NP-hardness is usually defined via mapping (Karp) reductions; Turing (Cook) reductions can blur NP vs co-NP distinctions.
5. *Approximation-preserving reductions?* -> L-reductions/PTAS-reductions transfer *inapproximability*, not just hardness.

8. Comparison Table

Aspect	Mapping (Karp) reduction	Turing (Cook) reduction
Form	one f, x in A iff f(x) in B	use B-oracle multiple times
Preserves	YES/NO directly	overall answer via computation
Used for	NP-completeness proofs	broader hardness
Strength	finer (keeps NP vs co-NP)	coarser
To prove...	Reduce...	
B is NP-hard	known NP-complete A to B (A $\leq_p$ B)	
B is in P	B to a known-easy problem	

9. Common Mistakes

- Reducing in the wrong direction (B to A instead of A to B) when proving B hard.
- Forgetting to prove the reduction runs in polynomial time.
- Proving only one direction of the "iff."
- Using a reduction that doesn't preserve the answer exactly.

10. Edge Cases

- The reduction must map NO-instances to NO-instances too (not just YES to YES).
- Poly-time includes producing a poly-size output (an exponential blow-up isn't allowed).
- Self-reductions and gadget correctness need careful case analysis.

11. How to Explain in Interview "A polynomial reduction from A to B is a poly-time function that turns any A-instance into a B-instance with the same yes/no answer. It means B is at least as hard as A - if B were easy, A would be too. So to prove a new problem B is NP-hard, you reduce a known NP-complete problem *to* B, never

the other way. Reductions are transitive, which is why proving SAT NP-complete once, via Cook-Levin, seeds hardness proofs for everything else."

12. Quick Revision

- $A \leq_p B$: poly-time answer-preserving map; $B \geq A$ in hardness.
- To prove B hard: reduce known-hard A TO B. Direction matters!
- $B \in P + A \leq_p B \Rightarrow A \in P$. Transitive.
- Prove NPC: in NP + reduce known NPC to it. Karp vs Turing reductions.

13. Practice Tasks

- Write the 3-SAT $\leq_p$ Clique reduction and prove both directions.
- Reduce Vertex Cover $\leq_p$ Independent Set (complement).
- Reduce Subset Sum $\leq_p$ Partition.

14. Cheat Sheet

- **Def**: Poly-time map turning A-instances into B-instances, preserving YES/NO.
- **Why**: spreads hardness; foundation of NP-completeness.
- **Most asked**: reduction direction; what $A \leq_p B$ implies.
- **Compare**: Karp (mapping) vs Cook (Turing) reductions.
- **One-liner**: "Transform A into B in poly time preserving the answer - proving B is at least as hard as A."

B13. Why is SAT Important?

1. Overview

- **Why SAT is special**: It was the **first problem proven NP-complete** (Cook-Levin, 1971), making it the *root* from which nearly every other NP-completeness proof descends. It's also the workhorse of practical constraint solving.
- **Why it matters**: SAT bridges deep theory (the birth of NP-completeness) and huge practice (industrial SAT solvers).
- **Why interviewers ask**: It ties together verifier, reductions, Cook-Levin, and real-world solvers in one question.

2. Core Idea

- **Intuition**: SAT is the "universal hard problem." Cook-Levin showed *every* NP problem's computation can be encoded as a boolean formula, so SAT captures all of NP. Once SAT was NP-complete, every other problem could be proven hard just by reducing SAT (or 3-SAT) to it - no need to re-encode Turing machines each time.
- **Analogy**: SAT is the "patient zero" of NP-completeness - the origin that infected all other problems with provable hardness via reductions.
- **Example**: To prove Clique, Vertex Cover, Graph Coloring, Subset Sum NP-complete, we reduce 3-SAT to each - all trace back to SAT.
- **Step-by-step (its dual importance)**: (theory) Cook-Levin $\Rightarrow$ SAT NP-complete $\Rightarrow$ seed for reductions; (practice) CDCL solvers $\Rightarrow$ SAT solved fast on real instances $\Rightarrow$ used everywhere.

3. Important Subtopics

- **Cook-Levin theorem:** encodes any NP verifier's run as a SAT formula. *Angle:* "Why is SAT the first NP-complete?"
- **Reduction seed:** transitivity spreads SAT's hardness to all NP-complete problems. *Angle:* role in proofs.
- **Practical SAT solvers (DPLL/CDCL):** solve million-variable instances despite NP-completeness. *Angle:* theory-vs-practice gap.
- **SMT extension:** SAT + theories (arithmetic, arrays) for program verification. *Angle:* modern relevance.

4. Real-World Example Hardware/software verification: Intel and others use SAT/SMT solvers to formally verify chip designs and detect bugs before manufacture (partly a response to the 1994 Pentium FDIV bug). SAT solvers also power AI planning, package dependency resolution, and automated theorem proving.

5. Mental Model

Theory: Cook-Levin => SAT is first NP-complete
 => reduce SAT (3-SAT) to prove others NP-complete
 => transitivity fans hardness across all of NP
 Practice: DPLL/CDCL solvers crush real-world instances
 => verification, planning, scheduling, EDA, crypto
 SAT = the bridge between "provably hard" and "practically solved."

6. Common Interview Questions

1. *Why is SAT important?* -> First NP-complete; root of all reductions; huge practical solver use. Mistake: only giving the theory half.
2. *What is Cook-Levin's significance?* -> Established SAT as NP-complete from first principles, seeding the theory.
3. *How does SAT help prove other problems hard?* -> Reduce SAT/3-SAT to them (transitivity).
4. *If SAT is NP-complete, how do solvers work?* -> Worst-case is exponential; real instances are structured, CDCL prunes.
5. *Why start reductions from SAT/3-SAT?* -> It's the canonical seed; 3-SAT gives clean gadgets.
6. *What if SAT were in P?* -> $P = NP$; all NP-complete problems become tractable.
7. *Where are SAT solvers used?* -> Verification, planning, dependency resolution, EDA, cryptanalysis.
8. *SAT vs SMT?* -> SMT adds theories; used in program verification.
9. *Is SAT theoretically or practically important?* -> Both - that's what makes it unique.
10. *What does SAT capture about NP?* -> The entire class - every NP computation encodes as a formula.

7. Deep-Dive Questions

1. *Sketch how Cook-Levin encodes a computation.* -> Variables describe the TM's tape/head/state at each time step; clauses enforce a valid start, valid transitions, and an accepting configuration - satisfiable iff the machine accepts.
2. *Why does one NP-complete problem suffice as a seed?* -> Reductions are transitive, so hardness proven once propagates to any problem SAT reduces to.
3. *Why do CDCL solvers scale despite NP-completeness?* -> Clause learning, unit propagation, and heuristics exploit real-world structure; NP-completeness only bounds the worst case.
4. *SAT's role in cryptography?* -> Reducing key-recovery to SAT tests cipher robustness; hardness assumptions underpin security.

5. *Phase transition in random SAT?* -> Near clause/variable ratio ~ 4.27 , instances flip from mostly-satisfiable to mostly-unsatisfiable and become hardest - a research touchstone.

8. Comparison Table

Aspect	SAT's importance	
Theory	first NP-complete (Cook-Levin), seed for all reductions	
Practice	industrial CDCL solvers solve huge instances	
Applications	verification, planning, EDA, dependency resolution, crypto	
Extensions	SMT (theories), MaxSAT, #SAT	

	SAT	Typical NP-complete problem
Discovery	proven from scratch (Cook-Levin)	proven via reduction from SAT
Role	seed / root	descendant

9. Common Mistakes

- Giving only the theoretical importance and missing the practical solver story (or vice versa).
- Thinking NP-completeness means SAT is unsolvable in practice.
- Forgetting that reductions from SAT are what make it foundational.

10. Edge Cases

- 2-SAT and Horn-SAT are in P - SAT's hardness is specific to general/3-CNF.
- Real-world instances are often far easier than worst case.
- SAT's importance is as much sociological (a shared reduction target) as mathematical.

11. How to Explain in Interview "SAT is important for two reasons. Theoretically, Cook-Levin proved it the first NP-complete problem by encoding any NP computation as a boolean formula, so SAT became the root - every other NP-completeness proof works by reducing SAT or 3-SAT to the target. Practically, modern CDCL SAT solvers routinely solve instances with millions of variables, powering chip verification, planning, and dependency resolution. SAT uniquely sits at the intersection of 'provably hard' and 'solved every day.'"

12. Quick Revision

- First NP-complete (Cook-Levin, 1971) - the reduction seed.
- Transitivity spreads its hardness to all NP-complete problems.
- Practically solved by DPLL/CDCL despite NP-completeness.
- Uses: verification, planning, EDA, dependency resolution, crypto. SMT extends it.

13. Practice Tasks

- Explain Cook-Levin's encoding idea in your own words.
- List 5 problems proven NP-complete via reduction from SAT/3-SAT.
- Install a SAT solver (e.g., MiniSat) and encode a small Sudoku.

14. Cheat Sheet

- **Def:** SAT = the first NP-complete problem; boolean satisfiability.
- **Why:** root of all NP-completeness proofs + backbone of practical solvers.
- **Most asked:** Cook-Levin; why it seeds reductions; why solvers work.
- **Compare:** SAT (proven from scratch) vs others (proven via reduction).
- **One-liner:** "The first NP-complete problem - every hardness proof reduces to it, and every industrial solver is built around it."

APPENDIX - MASTER CHEAT SHEETS

Chomsky Hierarchy (one table)

Type	Language	Grammar	Machine	Example
3	Regular	right/left-linear	Finite Automaton	a^*b^*
2	Context-Free	$A \rightarrow \alpha$	Pushdown Automaton	$a^n b^n$
1	Context-Sensitive	$\alpha A \beta \rightarrow \alpha \gamma \beta$	Linear-Bounded Automaton	$a^n b^n c^n$
0	Recursively Enumerable	unrestricted	Turing Machine	halting problem

Complexity landscape (one picture)

$P \subset NP \subset PSPACE \subset EXPTIME$
 NP-complete: hardest in NP (SAT, 3-SAT, clique, vertex cover, Hamiltonian, TSP-decision, subset sum, partition, knapsack-decision, 3-coloring)
 NP-hard: at least as hard as NP (adds TSP-optimization, halting, ...)
 Decidable (recursive) subset of Recognizable (RE) subset of all languages

"Which is in P vs NP-complete" traps (memorize)

In P (easy)	NP-complete (hard)
2-SAT	3-SAT
2-coloring (bipartite)	3-coloring
Euler circuit	Hamiltonian cycle
Fractional knapsack	0/1 knapsack
Shortest path	Longest path
Min spanning tree	TSP
Matching	Max independent set

Nondeterminism scorecard

Model	NFA/nondeterminism adds power?
Finite Automata	No (NFA = DFA)
Pushdown Automata	Yes (NPDA > DPDA)
Turing Machines	Open (P vs NP)

Weak vs strong NP-completeness

- **Weak** (pseudo-poly DP exists): Subset Sum, Partition, 0/1 Knapsack.
- **Strong** (hard even with small numbers): 3-Partition, TSP, most graph problems.